

LAPORAN TUGAS 7

SISTEM OPERASI



Dosen Pengampu:

Triawan Adi Cahyanto, M.Kom

Disusun Oleh:

Krisna Aji Pratama(2410651048)

Kelas C

PRODI TEKNIK INFORMATIKA

FAKULTAS TEKNIK

UNIVERSITAS MUHAMMADIYAH JEMBER

2025

DAFTAR ISI

BAB 1	3
PENDAHULUAN	3
1.1 LATAR BELAKANG	3
1.2 TUJUAN PRAKTIKUM	3
1.3 SPESIFIKASI SISTEM	4
BAB II	5
DASAR TEORI	5
2.1 MEMORY MANAGEMENT	5
2.2 CONTIGUOUS ALLOCATION	5
2.3 PAGING	6
2.4 PAGE TABLE STRUCTURES	6
2.5 SWAPPING	6
2.6 ARCHITECTURE	7
BAB III	8
JAWABAN TUGAS PRAKTIKUM	8
1.1 TUGAS 1	8
1.2 TUGAS 2	12
1.3 TUGAS 3	19
1.4 TUGAS 4	38
1.5 TUGAS 5	44
1.6 TUGAS 6	50
BAB IV	53
ANALISIS DAN PEMBAHASAN	53
4.1 ANALISIS TUGAS 1	53
4.2 ANALISIS TUGAS 2	56
4.3 ANALISIS TUGAS 3	57
4.4 ANALISIS TUGAS 4	60
4.5 ANALISIS TUGAS 5	62
4.6 ANALISIS TUGAS 6	63
BAB V	66
PENUTUP	66
DAFTAR PUSTAKA	67

BAB 1

PENDAHULUAN

1.1 LATAR BELAKANG

Manajemen memori merupakan komponen kritis dalam sistem operasi modern yang bertanggung jawab mengalokasikan, memantau, dan mengoptimalkan penggunaan memori fisik dan virtual. Dengan semakin kompleksnya aplikasi dan kebutuhan komputasi, pengelolaan memori yang efisien menjadi penentu utama performa sistem. Praktikum ini dirancang untuk memberikan pemahaman mendalam tentang konsep-konsep fundamental seperti alokasi memori kontigu, paging, swapping, dan struktur page table melalui implementasi langsung dalam bahasa pemrograman C. Melalui eksperimen praktis ini, mahasiswa dapat mengamati secara langsung bagaimana sistem operasi mengelola sumber daya memori yang terbatas untuk mendukung eksekusi berbagai proses secara simultan dan aman, sekaligus memahami trade-off antara performa, efisiensi, dan kompleksitas dalam desain sistem memori.

1.2 TUJUAN PRAKTIKUM

1. Menganalisis memory layout proses dan memahami perbedaan karakteristik setiap segmen (text, data, heap, stack)
2. Mengimplementasikan dan membandingkan algoritma alokasi memori kontigu (First-Fit, Best-Fit, Worst-Fit, Next-Fit) beserta analisis fragmentasi
3. Mensimulasikan sistem paging dengan berbagai algoritma page replacement (FIFO dan LRU) serta menganalisis fenomena Belady's Anomaly
4. Mengevaluasi berbagai struktur page table (multi-level dan inverted) dan mekanisme swapping dengan berbagai strategi pemilihan proses

1.3 SPESIFIKASI SISTEM

1. Sistem Operasi: Linux (distribusi Ubuntu/Debian) dengan kernel mendukung fitur memory management lengkap
2. Arsitektur Prosesor: x86-64 dengan dukungan virtualisasi dan mekanisme paging multi-level
3. Memori Fisik: Minimum 4 GB RAM dengan konfigurasi swap space terpisah
4. Perangkat Lunak: Compiler GCC versi 9 atau di atasnya, library standar C, serta tools diagnostik seperti lscpu, getconf, dan vmstat

BAB II

DASAR TEORI

2.1 MEMORY MANAGEMENT

Memory management adalah fungsi inti sistem operasi yang bertanggung jawab mengatur memori utama yakni RAM agar dapat digunakan secara aman dan efisien untuk berbagai proses. Sistem operasi melakukan address binding yang memetakan alamat logis program ke alamat fisik di RAM, serta memastikan isolasi memori antar proses untuk mencegah interferensi. Manajemen memori juga melibatkan alokasi dan dealokasi ruang memori sesuai kebutuhan proses, proteksi area memori, dan mekanisme sharing memori antar proses. Dalam praktikum ini, mahasiswa akan mempelajari layout memori (segmentasi: text, data, heap, stack) dan memahami bagaimana sistem operasi mengelola ruang alamat setiap proses melalui simulasi langsung dalam bahasa C.

2.2 CONTIGUOUS ALLOCATION

Contiguous allocation atau yang bisa disebut juga Alokasi memori berurutan adalah teknik di mana setiap proses ditempatkan di blok memori yang alamatnya bersebelahan. Metode ini menggunakan partisi tetap atau dinamis dan menerapkan algoritma seperti first-fit, best-fit, dan worst-fit untuk menentukan lokasi alokasi. First-fit memilih blok kosong pertama yang cukup besar, best-fit memilih blok terkecil yang memenuhi, sedangkan worst-fit memilih blok terbesar. Kelemahan utamanya adalah fragmentasi eksternal, yaitu terciptanya banyak celah kecil yang tidak terpakai sehingga mengurangi utilisasi memori. Dalam praktikum, mahasiswa akan mengimplementasikan ketiga algoritma tersebut dan menganalisis efisiensi serta fragmentasi yang dihasilkan.

2.3 PAGING

Paging adalah teknik manajemen memori yang membagi memori fisik menjadi blok ukuran tetap disebut frame, dan memori logis proses menjadi blok berukuran sama disebut page. Setiap page dapat dipetakan ke frame mana pun, sehingga menghilangkan masalah fragmentasi eksternal. Sistem operasi menggunakan page table untuk menerjemahkan alamat logis ke alamat fisik. Keuntungan paging termasuk fleksibilitas alokasi dan kemudahan dalam swapping. Namun, paging dapat menyebabkan fragmentasi internal dan overhead karena manajemen page table.

2.4 PAGE TABLE STRUCTURES

Struktur page table sangat berpengaruh pada performa dan efisiensi memori. Pada sistem sederhana, single-level page table dapat menghabiskan banyak memori. Hierarchical (multi-level) page table seperti dua atau tiga level mengurangi overhead dengan hanya mengalokasikan entri untuk ruang alamat yang digunakan. Hashed page table menggunakan fungsi hash untuk pencarian cepat, sementara inverted page table menyimpan satu entri per frame fisik, sehingga menghemat memori tetapi meningkatkan kompleksitas pencarian. Praktikum ini mengimplementasikan two-level page table dan menganalisis overhead memorinya.

2.5 SWAPPING

Swapping adalah mekanisme sistem operasi untuk memindahkan sementara proses dari memori utama ke penyimpanan sekunder (disk) ketika memori penuh, dan memuatnya kembali saat diperlukan. Proses swap out membebaskan ruang RAM dengan memindahkan proses idle ke swap space, sedangkan swap in mengembalikannya ke RAM. Swapping memungkinkan sistem menjalankan lebih banyak proses daripada kapasitas RAM, tetapi dapat menyebabkan thrashing jika terlalu sering terjadi. Strategi pemilihan proses yang akan di-swap (seperti FIFO, LRU, atau prioritas) sangat memengaruhi performa. Praktikum ini mensimulasikan mekanisme swapping dengan berbagai algoritma.

2.6 ARCHITECTURE

Arsitektur prosesor modern seperti Intel dan ARM memiliki karakteristik manajemen memori yang berbeda. Intel IA-32 (32-bit) menggunakan paging dua level dengan ruang alamat 4 GB, sedangkan x86-64 (64-bit) mendukung paging empat level dengan ruang alamat hingga 256 TB. ARMv8 mendukung paging multi-level dengan opsi ukuran halaman bervariasi (4KB, 16KB, 64KB). Setiap arsitektur memiliki struktur page table, ukuran TLB (Translation Lookaside Buffer), dan mekanisme proteksi memori yang unik. Memahami perbedaan ini penting untuk pengembangan sistem yang optimal. Praktikum menganalisis spesifikasi memori melalui perintah sistem dan simulasi perbandingan arsitektur.

BAB III

JAWABAN TUGAS PRAKTIKUM

1.1 TUGAS 1

3.1.1 Deskripsi Tugas

Tugas 1 bertujuan untuk menganalisis memory layout pada sistem operasi melalui modifikasi program `address_demo.c`. Mahasiswa diminta menambahkan beberapa fitur, meliputi pembuatan array statis dan dinamis untuk mengamati alokasi memori, penambahan tiga fungsi berbeda guna membandingkan alamat fungsi pada text segment, implementasi fungsi rekursif untuk menampilkan perubahan alamat stack di setiap level pemanggilan, serta penampilan alamat string literal. Selanjutnya dilakukan analisis jarak antar segmen memori, pengamatan konsistensi alamat melalui eksekusi berulang, penjelasan pengaruh ASLR, serta pencocokan alamat dengan memory map beserta permission tiap segmen.

3.1.2 Source Code

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>

int main();

int global_var = 100;
int static_array[3] = {10,20,30};
char *text = "Main Memory Praktikum";

void func1() {}
void func2() {}
void func3() {}

void recursive(int level){
    int x = level;
    printf(" Level %d local_var address: %p value: %d\n",
        level, &x, x);

    if(level < 2)
        recursive(level + 1);
}

void print_memory(){
    int local = 5;
    int *heap1 = malloc(sizeof(int));
    int *heap2 = malloc(sizeof(int));
    int *dyn = malloc(3 * sizeof(int));

    *heap1 = 50;
    *heap2 = 75;
```



```

dyn[0] = 1;

printf("\nENHANCED MEMORY LAYOUT\n\n");

printf("1. Text Segment:\n");
printf("  main() : %p\n", main);
printf("  func1() : %p\n", func1);
printf("  func2() : %p\n", func2);
printf("  func3() : %p\n\n", func3);

printf("2. Data Segment:\n");
printf("  global_var address: %p value: %d\n", &global_var, global_var);
printf("  static_array[0] address: %p value: %d\n", &static_array[0], static_array[0]);
printf("  string literal address: %p value: %s\n\n", text, text);

printf("3. Heap:\n");
printf("  heap1 address: %p value: %d\n", heap1, *heap1);
printf("  heap2 address: %p value: %d\n", heap2, *heap2);
printf("  dyn_array[0] address: %p value: %d\n\n", &dyn[0], dyn[0]);

printf("4. Stack:\n");
printf("  local address: %p value: %d\n\n", &local, local);

printf("4. Stack (Recursive Calls):\n");
recursive(0);

free(heap1);
free(heap2);
free(dyn);
}

int main(){
  printf("PID: %d\n", getpid());
  print_memory();

  printf("\nTekan Enter untuk melihat memory map...");
  getchar();

  system("cat /proc/self/maps");
  return 0;
}

```


- Linking

Linker menggabungkan object file dengan library C standar, menyelesaikan referensi ke fungsi eksternal seperti `printf()`, `malloc()`, `free()`, dan `system()`. Hasilnya adalah sebuah executable file yang siap dijalankan oleh sistem operasi.

- Loading

Saat executable dijalankan, sistem operasi memuat program ke dalam memori virtual proses dengan pembagian segmen:

- Text Segment: berisi kode instruksi fungsi `main()`, `print_memory()`, `recursive()`, serta `func1()`, `func2()`, dan `func3()`.
- Data Segment: menyimpan variabel global terinisialisasi seperti `global_var`, `static_array`, serta pointer text yang menunjuk ke string literal "Main Memory Praktikum".

- Stack Setup

Sistem operasi membuat stack frame awal untuk fungsi `main()`. Stack digunakan untuk menyimpan variabel lokal, parameter fungsi, dan alamat pengembalian setiap pemanggilan fungsi.

- Main Execution

Fungsi `main()` mulai dieksekusi. Program pertama-tama menampilkan PID (Process ID) menggunakan `getpid()`, yang menunjukkan identitas unik proses dalam sistem operasi.

- Function Call

Fungsi `print_memory()` dipanggil dari `main()`. Pemanggilan ini menyebabkan dibuatnya stack frame baru, yang berisi variabel lokal dan pointer heap milik fungsi tersebut.

- Heap Allocation

Di dalam `print_memory()`, program memanggil `malloc()` sebanyak tiga kali untuk mengalokasikan memori dinamis pada heap (`heap1`, `heap2`, dan `dyn`). Hal ini menunjukkan bahwa heap dikelola secara manual dan berada terpisah dari stack.

- Memory Layout Display

Program mencetak alamat dan nilai dari berbagai segmen memori:

- Text Segment: alamat fungsi `main()`, `func1()`, `func2()`, dan `func3()`
- Data Segment: alamat dan nilai `global_var`, `static_array`, serta string literal
- Heap: alamat hasil alokasi `malloc()`
- Stack: alamat variabel lokal dalam `print_memory()`

- Recursive Demonstration

Fungsi `recursive()` dipanggil secara bertingkat. Setiap pemanggilan rekursif menciptakan stack frame baru dengan alamat variabel lokal `x` yang berbeda, membuktikan bahwa stack bekerja dengan mekanisme LIFO (Last In First Out).

- Memory Release

Setelah penggunaan selesai, semua memori heap dilepas menggunakan free(). Hal ini menunjukkan praktik manajemen memori yang benar untuk mencegah memory leak.

- Memory Mapping

Program menunggu input pengguna, kemudian menampilkan isi file /proc/self/maps. Informasi ini menunjukkan peta memori aktual proses, termasuk text, data, heap, stack, shared libraries, dan region lainnya yang dikelola oleh kernel Linux.

- Termination

Setelah main() selesai dieksekusi, proses berakhir. Sistem operasi secara otomatis membersihkan seluruh resource yang digunakan oleh program, termasuk stack, heap, dan file descriptor.

1.2 TUGAS 2

1.2.1 Deskripsi Tugas

Tugas 2 berfokus pada pengembangan dan analisis program memory allocator dengan menambahkan algoritma Next-Fit sebagai perluasan dari First-Fit, di mana pencarian blok memori dimulai dari posisi alokasi terakhir. Selain itu, mahasiswa diminta mengimplementasikan fitur perhitungan fragmentasi untuk mengevaluasi efisiensi penggunaan memori. Selanjutnya dilakukan benchmark testing untuk membandingkan empat algoritma alokasi, yaitu First-Fit, Best-Fit, Worst-Fit, dan Next-Fit, berdasarkan waktu eksekusi, tingkat fragmentasi, serta kegagalan alokasi. Hasil pengujian dianalisis dalam bentuk tabel dan grafik guna menilai performa, efisiensi, dan kondisi optimal penggunaan masing-masing algoritma.

3.2.2 Source Code

```
#include <stdio.h>
#include <time.h>

#define MAX_BLOCKS 100

typedef struct {
    int size;
    int free;    // 1 = free, 0 = allocated
    int process_id;
} Block;

Block memory[MAX_BLOCKS];
int block_count = 1;
int last_alloc_index = 0;

void init_memory(int total_size) {
```

```

memory[0].size = total_size;
memory[0].free = 1;
memory[0].process_id = -1;
block_count = 1;
last_alloc_index = 0;
}

void display_memory() {
    printf("\nMemory State:\n");
    for (int i = 0; i < block_count; i++) {
        if (memory[i].free)
            printf("Block %d: FREE | Size = %d\n", i, memory[i].size);
        else
            printf("Block %d: P%d | Size = %d\n", i, memory[i].process_id, memory[i].size);
    }
}

void deallocate(int pid) {
    for (int i = 0; i < block_count; i++) {
        if (!memory[i].free && memory[i].process_id == pid) {
            memory[i].free = 1;
            memory[i].process_id = -1;
            return;
        }
    }
}

int first_fit(int pid, int size) {
    for (int i = 0; i < block_count; i++) {
        if (memory[i].free && memory[i].size >= size) {

            if (memory[i].size > size) {
                for (int j = block_count; j > i + 1; j--)
                    memory[j] = memory[j - 1];

                memory[i + 1].size = memory[i].size - size;
                memory[i + 1].free = 1;
                memory[i + 1].process_id = -1;
                block_count++;
            }

            memory[i].size = size;
            memory[i].free = 0;
            memory[i].process_id = pid;
            return 1;
        }
    }
    return 0;
}

```

```

int best_fit(int pid, int size) {
    int best = -1;

    for (int i = 0; i < block_count; i++) {
        if (memory[i].free && memory[i].size >= size) {
            if (best == -1 || memory[i].size < memory[best].size)
                best = i;
        }
    }

    if (best == -1) return 0;

    if (memory[best].size > size) {
        for (int j = block_count; j > best + 1; j--)
            memory[j] = memory[j - 1];

        memory[best + 1].size = memory[best].size - size;
        memory[best + 1].free = 1;
        memory[best + 1].process_id = -1;
        block_count++;
    }

    memory[best].size = size;
    memory[best].free = 0;
    memory[best].process_id = pid;
    return 1;
}

int worst_fit(int pid, int size) {
    int worst = -1;

    for (int i = 0; i < block_count; i++) {
        if (memory[i].free && memory[i].size >= size) {
            if (worst == -1 || memory[i].size > memory[worst].size)
                worst = i;
        }
    }

    if (worst == -1) return 0;

    if (memory[worst].size > size) {
        for (int j = block_count; j > worst + 1; j--)
            memory[j] = memory[j - 1];

        memory[worst + 1].size = memory[worst].size - size;
        memory[worst + 1].free = 1;
        memory[worst + 1].process_id = -1;
        block_count++;
    }
}

```

```

memory[worst].size = size;
memory[worst].free = 0;
memory[worst].process_id = pid;
return 1;
}

int next_fit(int pid, int size) {
    int i = last_alloc_index;
    int scanned = 0;

    while (scanned < block_count) {
        if (memory[i].free && memory[i].size >= size) {

            if (memory[i].size > size) {
                for (int j = block_count; j > i + 1; j--)
                    memory[j] = memory[j - 1];

                memory[i + 1].size = memory[i].size - size;
                memory[i + 1].free = 1;
                memory[i + 1].process_id = -1;
                block_count++;
            }

            memory[i].size = size;
            memory[i].free = 0;
            memory[i].process_id = pid;

            last_alloc_index = i;
            return 1;
        }

        i = (i + 1) % block_count;
        scanned++;
    }
    return 0;
}

void calculate_fragmentation() {
    int total_free = 0, largest = 0, holes = 0;

    for (int i = 0; i < block_count; i++) {
        if (memory[i].free) {
            total_free += memory[i].size;
            holes++;
            if (memory[i].size > largest)
                largest = memory[i].size;
        }
    }

    float ext = (total_free > 0) ?

```

```

        (1.0 - (float)largest / total_free) * 100 : 0;

    printf("Total Free Space    : %d\n", total_free);
    printf("Largest Free Block   : %d\n", largest);
    printf("Free Holes           : %d\n", holes);
    printf("External Fragmentation: %.2f%%\n", ext);
}

void benchmark(const char *name, int (*alloc)(int,int)) {
    init_memory(1000);
    clock_t start = clock();

    alloc(1,100);
    alloc(2,200);
    alloc(3,300);
    alloc(4,250);
    alloc(5,150);

    deallocate(2);
    deallocate(4);

    alloc(6,180);
    alloc(7,220);
    alloc(8,100);

    clock_t end = clock();

    printf("\n %s \n", name);
    printf("Execution Time: %.2f us\n",
        (double)(end-start)*1e6/CLOCKS_PER_SEC);

    calculate_fragmentation();
    display_memory();
}

int main() {
    benchmark("First Fit", first_fit);
    benchmark("Best Fit", best_fit);
    benchmark("Worst Fit", worst_fit);
    benchmark("Next Fit", next_fit);
    return 0;
}

```


3.2.3 Screenshot Output

[illegible]

```

Worst Fit
Execution Time: 3.00 us
Total Free Space      : 170
Largest Free Block    : 150
Free Holes            : 2
External Fragmentation: 41.18%

Memory State:
Block 0: P1 | Size = 100
Block 1: P0 | Size = 100
Block 2: FREE | Size = 100
Block 3: P3 | Size = 100
Block 4: P6 | Size = 100
Block 5: FREE | Size = 70
Block 6: P5 | Size = 150

Next Fit
Execution Time: 3.00 us
Total Free Space      : 50
Largest Free Block    : 30
Free Holes            : 2
External Fragmentation: 48.00%

Memory State:
Block 0: P1 | Size = 100
Block 1: P6 | Size = 100
Block 2: FREE | Size = 70
Block 3: P3 | Size = 100
Block 4: P7 | Size = 120
Block 5: FREE | Size = 30
Block 6: P5 | Size = 150

```

3.2.4 Penjelasan Implementasi

- Preprocessing

Pada tahap preprocessing, compiler memproses header `stdio.h` dan `time.h`. Header `stdio.h` digunakan untuk operasi input-output seperti `printf()`, sedangkan `time.h` digunakan untuk pengukuran waktu eksekusi melalui fungsi `clock()`. Selain itu, konstanta `MAX_BLOCKS` didefinisikan sebagai batas maksimum jumlah blok memori yang dapat dikelola oleh sistem simulasi.

- Compilation

Pada tahap kompilasi, kode C diterjemahkan menjadi assembly code. Compiler memeriksa struktur data Block, logika pengulangan, percabangan, serta pemanggilan fungsi pointer (function pointer) pada fungsi benchmark(). Implementasi algoritma alokasi memori seperti first_fit, best_fit, worst_fit, dan next_fit dianalisis secara sintaks dan semantik.

- Assembly

Kode assembly hasil kompilasi kemudian diubah menjadi object file yang berisi machine code. Object file ini mencakup instruksi CPU, struktur data, dan tabel simbol, namun belum dapat dieksekusi karena belum melalui proses linking.

- Linking

Pada tahap linking, object file digabungkan dengan library standar C. Referensi fungsi seperti printf() dan clock() diselesaikan sehingga dihasilkan sebuah executable program yang siap dijalankan.

- Loading

Saat program dieksekusi, sistem operasi memuat executable ke dalam memori. Segmen text berisi instruksi program, sedangkan segmen data menyimpan variabel global seperti array memory, block_count, dan last_alloc_index yang digunakan untuk menyimulasikan manajemen memori.

- Memory Initialization

Fungsi init_memory() dipanggil untuk menginisialisasi memori awal sebagai satu blok bebas berukuran total tertentu. Pada tahap ini, memori dipandang sebagai ruang kontigu tunggal yang siap dialokasikan ke proses-proses.

- Allocation Algorithms Execution

Program menguji empat algoritma alokasi memori:

- First Fit: memilih blok bebas pertama yang cukup besar
- Best Fit: memilih blok bebas terkecil yang masih dapat menampung proses
- Worst Fit: memilih blok bebas terbesar
- Next Fit: melanjutkan pencarian dari posisi alokasi terakhir

Setiap algoritma bekerja dengan mekanisme block splitting, yaitu memecah blok bebas menjadi blok terpakai dan sisa blok bebas.

- Deallocation Process

Fungsi deallocate() membebaskan memori berdasarkan process_id. Blok yang dilepas ditandai sebagai bebas, namun tidak dilakukan penggabungan (coalescing), sehingga potensi fragmentasi eksternal tetap ada.

- Fragmentation Analysis

Fungsi calculate_fragmentation() menghitung:

- Total ruang bebas
- Ukuran blok bebas terbesar
- Jumlah hole (lubang memori)
- Persentase fragmentasi eksternal

Perhitungan ini menunjukkan efisiensi masing-masing algoritma dalam memanfaatkan ruang memori.

- Benchmarking

Fungsi `benchmark()` digunakan untuk mengukur waktu eksekusi setiap algoritma menggunakan `clock()`. Penggunaan function pointer memungkinkan algoritma diuji dengan skenario yang sama, sehingga perbandingan performa menjadi objektif.

- Memory State Display

Fungsi `display_memory()` menampilkan kondisi akhir memori, termasuk status setiap blok (FREE atau teralokasi oleh proses tertentu) dan ukurannya. Output ini memberikan visualisasi nyata tentang fragmentasi yang dihasilkan oleh tiap algoritma.

- Termination

Setelah seluruh algoritma diuji, fungsi `main()` selesai dieksekusi dan program berakhir. Sistem operasi kemudian membersihkan seluruh resource yang digunakan oleh program.

1.3 TUGAS 3

1.3.1 Deskripsi Tugas

Tugas 3 bertujuan mengembangkan *advanced paging system* dengan menambahkan mekanisme *page replacement* dan analisis kinerja. Mahasiswa diminta memodifikasi `paging_simulator.c` agar mampu menangani kondisi ketika seluruh frame telah penuh dengan mengimplementasikan algoritma FIFO dan LRU. Sistem harus mencatat statistik penting seperti jumlah page fault, hit ratio, serta disk writes untuk halaman bertipe *dirty*. Selanjutnya dilakukan pengujian menggunakan reference string tertentu dengan 4 frame, lalu dibandingkan hasil FIFO dan LRU. Analisis dilengkapi dengan tabel *trace*, pengujian *Belady's Anomaly* pada variasi jumlah frame, serta evaluasi performa untuk menentukan keunggulan masing-masing algoritma.

3.3.2 Source Code

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <math.h>
#include <time.h>
#include <limits.h>

#define FRAME_SIZE 256
#define MAX_FRAMES 16
#define PAGE_SIZE FRAME_SIZE

// Algoritma yang tersedia
typedef enum {
    FIFO,
```

```

    LRU
} Algorithm;

typedef struct {
    int page_number;
    int frame_number;
    int valid;
    int reference_bit;
    int modify_bit;
    int last_used; // Untuk LRU - timestamp penggunaan terakhir
} PageTableEntry;

typedef struct {
    int occupied;
    int page_number;
    int load_time; // Untuk FIFO - waktu pemuatan halaman
    int dirty; // Untuk statistik - halaman telah dimodifikasi
} Frame;

// Variabel global
Frame physical_memory[MAX_FRAMES];
PageTableEntry page_table[100];
int num_pages = 0;
int num_frames = 4; // Default 4 frames untuk test scenario
Algorithm current_algorithm = FIFO;

// Statistik
int page_faults = 0;
int page_hits = 0;
int disk_writes = 0;
int total_accesses = 0;

// Untuk FIFO
int fifo_queue[MAX_FRAMES];
int front = 0, rear = -1, queue_count = 0;

// Untuk LRU
int lru_counter = 0;

void initialize_system(int frames) {
    num_frames = frames;

    // Reset memory frames
    for (int i = 0; i < MAX_FRAMES; i++) {
        physical_memory[i].occupied = 0;
        physical_memory[i].page_number = -1;
        physical_memory[i].load_time = 0;
        physical_memory[i].dirty = 0;
    }
}

```

```

// Reset page table
for (int i = 0; i < 100; i++) {
    page_table[i].page_number = i;
    page_table[i].frame_number = -1;
    page_table[i].valid = 0;
    page_table[i].reference_bit = 0;
    page_table[i].modify_bit = 0;
    page_table[i].last_used = -1;
}

// Reset FIFO queue
front = 0;
rear = -1;
queue_count = 0;

// Reset LRU counter
lru_counter = 0;

// Reset statistik
page_faults = 0;
page_hits = 0;
disk_writes = 0;
total_accesses = 0;
}

void display_page_table() {
    printf("\n=== PAGE TABLE ===\n");
    printf("%-12s %-15s %-10s %-12s %-12s %-12s\n",
           "Page", "Frame", "Valid", "Referenced", "Modified", "Last Used");
    printf("-----\n");

    for (int i = 0; i < num_pages; i++) {
        printf("%-12d %-15d %-10s %-12s %-12s %-12d\n",
               page_table[i].page_number,
               page_table[i].frame_number,
               page_table[i].valid ? "Yes" : "No",
               page_table[i].reference_bit ? "Yes" : "No",
               page_table[i].modify_bit ? "Yes" : "No",
               page_table[i].last_used);
    }
    printf("\n");
}

void display_physical_memory() {
    printf("=== PHYSICAL MEMORY (%d FRAMES) ===\n", num_frames);
    printf("%-10s %-10s %-15s %-10s %-10s\n",
           "Frame", "Status", "Page Number", "Load Time", "Dirty");
    printf("-----\n");

    for (int i = 0; i < num_frames; i++) {

```

```

        printf("%-10d %-10s", i, physical_memory[i].occupied ? "Occupied" : "Free");
        if (physical_memory[i].occupied) {
            printf("%-15d %-10d %-10s",
                physical_memory[i].page_number,
                physical_memory[i].load_time,
                physical_memory[i].dirty ? "Yes" : "No");
        }
        printf("\n");
    }
    printf("\n");
}

int find_free_frame() {
    for (int i = 0; i < num_frames; i++) {
        if (!physical_memory[i].occupied) {
            return i;
        }
    }
    return -1;
}

void enqueue_fifo(int page_num) {
    rear = (rear + 1) % num_frames;
    fifo_queue[rear] = page_num;
    if (queue_count < num_frames) {
        queue_count++;
    }
}

int dequeue_fifo() {
    if (queue_count == 0) return -1;
    int page_num = fifo_queue[front];
    front = (front + 1) % num_frames;
    queue_count--;
    return page_num;
}

int find_lru_victim() {
    int victim_frame = -1;
    int min_last_used = INT_MAX;

    for (int i = 0; i < num_frames; i++) {
        if (physical_memory[i].occupied) {
            int page_num = physical_memory[i].page_number;
            if (page_table[page_num].last_used < min_last_used) {
                min_last_used = page_table[page_num].last_used;
                victim_frame = i;
            }
        }
    }
}

```

```

    return victim_frame;
}

int find_fifo_victim() {
    // FIFO menggunakan queue, jadi kita ambil dari depan
    if (queue_count == 0) return -1;

    int victim_page = fifo_queue[front];
    for (int i = 0; i < num_frames; i++) {
        if (physical_memory[i].occupied && physical_memory[i].page_number ==
victim_page) {
            return i;
        }
    }
    return -1;
}

int replace_page(int page_num, int write) {
    int victim_frame = -1;

    if (current_algorithm == FIFO) {
        victim_frame = find_fifo_victim();
    } else if (current_algorithm == LRU) {
        victim_frame = find_lru_victim();
    }

    if (victim_frame == -1) {
        return -1;
    }

    int victim_page = physical_memory[victim_frame].page_number;

    // Jika halaman korban telah dimodifikasi, perlu write ke disk
    if (page_table[victim_page].modify_bit && physical_memory[victim_frame].dirty) {
        disk_writes++;
        printf(" [DISK WRITE] Page %d (dirty) ditulis ke disk\n", victim_page);
    }

    // Update page table untuk halaman korban
    page_table[victim_page].valid = 0;
    page_table[victim_page].frame_number = -1;
    page_table[victim_page].modify_bit = 0;

    // Update page table untuk halaman baru
    page_table[page_num].frame_number = victim_frame;
    page_table[page_num].valid = 1;
    page_table[page_num].reference_bit = 1;
    page_table[page_num].last_used = lru_counter++;
    if (write) {
        page_table[page_num].modify_bit = 1;
    }
}

```

```

}

// Update physical memory
physical_memory[victim_frame].page_number = page_num;
physical_memory[victim_frame].load_time = total_accesses;
physical_memory[victim_frame].dirty = write ? 1 : 0;

// Update FIFO queue jika menggunakan FIFO
if (current_algorithm == FIFO) {
    dequeue_fifo();
    enqueue_fifo(page_num);
}

return victim_frame;
}

int allocate_page(int page_num, int write) {
    total_accesses++;

    if (page_num >= 100) {
        printf("Error: Page number terlalu besar\n");
        return -1;
    }

    // Jika halaman sudah ada di memori (HIT)
    if (page_table[page_num].valid) {
        page_hits++;
        page_table[page_num].reference_bit = 1;
        page_table[page_num].last_used = lru_counter++;

        if (write) {
            page_table[page_num].modify_bit = 1;
            // Update dirty bit di physical memory
            int frame_num = page_table[page_num].frame_number;
            if (frame_num >= 0 && frame_num < num_frames) {
                physical_memory[frame_num].dirty = 1;
            }
        }

        return page_table[page_num].frame_number;
    }

    // PAGE FAULT
    page_faults++;

    int frame = find_free_frame();

    if (frame == -1) {
        // Tidak ada frame kosong, perlu page replacement
        printf(" [PAGE REPLACEMENT] Menggunakan algoritma %s\n",

```



```

        current_algorithm == FIFO ? "FIFO" : "LRU");
    frame = replace_page(page_num, write);
    if (frame == -1) {
        printf("Error: Gagal melakukan page replacement\n");
        return -1;
    }
    printf(" Page %d menggantikan page di frame %d\n", page_num, frame);
} else {
    // Ada frame kosong
    page_table[page_num].frame_number = frame;
    page_table[page_num].valid = 1;
    page_table[page_num].reference_bit = 1;
    page_table[page_num].last_used = lru_counter++;
    if (write) {
        page_table[page_num].modify_bit = 1;
    }

    physical_memory[frame].occupied = 1;
    physical_memory[frame].page_number = page_num;
    physical_memory[frame].load_time = total_accesses;
    physical_memory[frame].dirty = write ? 1 : 0;

    // Untuk FIFO, tambahkan ke queue
    if (current_algorithm == FIFO) {
        enqueue_fifo(page_num);
    }

    printf("Page %d dialokasikan ke frame %d\n", page_num, frame);
}

if (page_num >= num_pages) {
    num_pages = page_num + 1;
}

return frame;
}

void display_statistics() {
    printf("\n=== STATISTIK ===\n");
    printf("Algoritma: %s\n", current_algorithm == FIFO ? "FIFO" : "LRU");
    printf("Jumlah Frames: %d\n", num_frames);
    printf("Total Akses: %d\n", total_accesses);
    printf("Page Faults: %d\n", page_faults);
    printf("Page Hits: %d\n", page_hits);

    if (total_accesses > 0) {
        float fault_rate = (float)page_faults / total_accesses * 100;
        float hit_rate = (float)page_hits / total_accesses * 100;
        printf("Page Fault Rate: %.2f%%\n", fault_rate);
        printf("Hit Rate: %.2f%%\n", hit_rate);
    }
}

```

```

    }

    printf("Disk Writes: %d\n", disk_writes);
    printf("\n");
}

void run_test_scenario(int frames, Algorithm algo) {

    printf("\n=====
===== \n");
    printf("|| TEST SCENARIO - %s dengan %d frames || \n",
           algo == FIFO ? "FIFO" : "LRU", frames);

    printf("=====
===== \n\n");

    initialize_system(frames);
    current_algorithm = algo;

    // Reference string dari soal
    int reference_string[] = {7,0,1,2,0,3,0,4,2,3,0,3,2,1,2,0,1,7,0,1};
    int length = sizeof(reference_string) / sizeof(reference_string[0]);

    // Untuk menampilkan trace table
    printf("Step\tReference\tFrame 0\tFrame 1\tFrame 2\tFrame 3\tFault?\n");
    printf("-----\n");

    for (int i = 0; i < length; i++) {
        int page_num = reference_string[i];
        int fault_before = page_faults;

        // Simulasi akses read
        allocate_page(page_num, 0);

        // Tampilkan trace
        printf("%d\t%d\t\t", i+1, page_num);

        // Tampilkan isi frames
        for (int f = 0; f < frames; f++) {
            if (physical_memory[f].occupied) {
                printf("%d\t", physical_memory[f].page_number);
            } else {
                printf("-\t");
            }
        }

        // Tampilkan apakah terjadi fault
        if (page_faults > fault_before) {
            printf("FAULT");
        } else {

```

```

        printf("HIT");
    }
    printf("\n");
}

display_statistics();
}

void compare_algorithms() {

printf("\n=====
=====
\");
printf("|| PERBANDINGAN ALGORITMA (4 frames) ||\n");

printf("||
=====
=====
\");

printf("%-15s %-20s %-15s %-20s\n",
    "Algoritma", "Jumlah Page Fault", "Hit Ratio", "Disk Writes");
printf("-----\n");

// Jalankan FIFO
run_test_scenario(4, FIFO);
int fifo_faults = page_faults;
int fifo_writes = disk_writes;
float fifo_hit_rate = (float)page_hits / total_accesses * 100;

// Jalankan LRU
run_test_scenario(4, LRU);
int lru_faults = page_faults;
int lru_writes = disk_writes;
float lru_hit_rate = (float)page_hits / total_accesses * 100;

printf("\n=== TABEL PERBANDINGAN ===\n");
printf("%-15s %-20d %-15.2f%% %-20d\n",
    "FIFO", fifo_faults, fifo_hit_rate, fifo_writes);
printf("%-15s %-20d %-15.2f%% %-20d\n",
    "LRU", lru_faults, lru_hit_rate, lru_writes);
}

void belady_anomaly_test() {

printf("\n=====
=====
\");
printf("|| BELADY'S ANOMALY TEST ||\n");

printf("||
=====
=====
\");

printf("Menguji apakah page fault berkurang seiring bertambahnya frames:\n\n");

```

```

// Test untuk 3, 4, dan 5 frames
int frames_list[] = {3, 4, 5};
int num_tests = sizeof(frames_list) / sizeof(frames_list[0]);

printf("%-10s %-15s %-15s\n", "Frames", "FIFO Faults", "LRU Faults");
printf("-----\n");

for (int i = 0; i < num_tests; i++) {
    int frames = frames_list[i];

    // Test FIFO
    run_test_scenario(frames, FIFO);
    int fifo_faults = page_faults;

    // Test LRU
    run_test_scenario(frames, LRU);
    int lru_faults = page_faults;

    printf("%-10d %-15d %-15d\n", frames, fifo_faults, lru_faults);
}

printf("\nANALISIS:\n");
printf("Belady's Anomaly terjadi ketika jumlah page fault BERTAMBAH\n");
printf("meskipun jumlah frame ditambah. Fenomena ini bisa terjadi\n");
printf("pada algoritma FIFO tetapi tidak pada LRU.\n");
}

void demo_scenario() {
    printf("\n");
    printf("|| DEMO SCENARIO: PAGING SYSTEM ||\n");
    printf("||\n");

    initialize_system(4);
    current_algorithm = FIFO;

    printf("Skenario: Program berukuran 2048 bytes\n");
    printf("Jumlah pages: %d pages\n\n", 2048 / PAGE_SIZE);

    // Simulasi beberapa akses
    int accesses[][2] = {
        {0, 0}, // Read address 0
        {300, 1}, // Write address 300
        {512, 0}, // Read address 512
        {100, 0}, // Read address 100
        {1024, 1} // Write address 1024
    };

    for (int i = 0; i < 5; i++) {
        int addr = accesses[i][0];

```

```

int write = accesses[i][1];

printf("\nAkses %s pada logical address %d:\n",
       write ? "WRITE" : "READ", addr);

int page_num = addr / PAGE_SIZE;
int frame = allocate_page(page_num, write);

if (frame != -1) {
    printf(" Page %d -> Frame %d\n", page_num, frame);
}
}

display_page_table();
display_physical_memory();
display_statistics();
}

int main() {
    int choice, address, write, frames, algo_choice;

    // Default initialization
    initialize_system(4);

    while(1) {
        printf("\n=== ADVANCED PAGING SIMULATOR ===\n");
        printf("1. Set Jumlah Frames\n");
        printf("2. Set Algoritma (FIFO/LRU)\n");
        printf("3. Allocate Page\n");
        printf("4. Access Memory (Read)\n");
        printf("5. Access Memory (Write)\n");
        printf("6. Display Page Table\n");
        printf("7. Display Physical Memory\n");
        printf("8. Display Statistics\n");
        printf("9. Run Demo Scenario\n");
        printf("10. Run Test Scenario (Reference String)\n");
        printf("11. Compare FIFO vs LRU\n");
        printf("12. Belady's Anomaly Test\n");
        printf("13. Reset System\n");
        printf("14. Exit\n");
        printf("Pilihan: ");
        scanf("%d", &choice);

        switch(choice) {
            case 1:
                printf("Masukkan jumlah frames (1-%d): ", MAX_FRAMES);
                scanf("%d", &frames);
                if (frames > 0 && frames <= MAX_FRAMES) {
                    initialize_system(frames);
                    printf("Sistem direset dengan %d frames\n", frames);
                }
            }
        }
    }
}

```

```
    } else {  
        printf("Jumlah frames tidak valid!\n");  
    }  
    break;
```

case 2:

```
    printf("Pilih algoritma:\n");  
    printf("1. FIFO\n");  
    printf("2. LRU\n");  
    printf("Pilihan: ");  
    scanf("%d", &algo_choice);  
    if (algo_choice == 1) {  
        current_algorithm = FIFO;  
        printf("Algoritma diubah menjadi FIFO\n");  
    } else if (algo_choice == 2) {  
        current_algorithm = LRU;  
        printf("Algoritma diubah menjadi LRU\n");  
    } else {  
        printf("Pilihan tidak valid!\n");  
    }  
    break;
```

case 3:

```
    printf("Masukkan page number: ");  
    scanf("%d", &address);  
    printf("Tipe akses (0=Read, 1=Write): ");  
    scanf("%d", &write);  
    allocate_page(address, write);  
    break;
```

case 4:

```
    printf("Masukkan logical address: ");  
    scanf("%d", &address);  
    printf("Page %d diakses (READ)\n", address / PAGE_SIZE);  
    allocate_page(address / PAGE_SIZE, 0);  
    break;
```

case 5:

```
    printf("Masukkan logical address: ");  
    scanf("%d", &address);  
    printf("Page %d diakses (WRITE)\n", address / PAGE_SIZE);  
    allocate_page(address / PAGE_SIZE, 1);  
    break;
```

case 6:

```
    display_page_table();  
    break;
```

case 7:

```
    display_physical_memory();
```

```

        break;

    case 8:
        display_statistics();
        break;

    case 9:
        demo_scenario();
        break;

    case 10:
        printf("Masukkan jumlah frames untuk test: ");
        scanf("%d", &frames);
        printf("Pilih algoritma (1=FIFO, 2=LRU): ");
        scanf("%d", &algo_choice);
        if (frames > 0 && frames <= MAX_FRAMES && (algo_choice == 1 ||
algo_choice == 2)) {
            run_test_scenario(frames, algo_choice == 1 ? FIFO : LRU);
        } else {
            printf("Input tidak valid!\n");
        }
        break;

    case 11:
        compare_algorithms();
        break;

    case 12:
        belady_anomaly_test();
        break;

    case 13:
        initialize_system(num_frames);
        printf("Sistem direset\n");
        break;

    case 14:
        printf("Keluar dari program...\n");
        return 0;

    default:
        printf("Pilihan tidak valid!\n");
    }
}

return 0;
}

```

3.3.3 Screenshot Output

[illegible]

→ ALGORITMA POLISH EASYMAN v0.0

1. Hit Judikan Frame
2. Hit Algoritma (FIFO/LRU)
3. Algoritma Page
4. Access Memory Demand
5. Access Memory Location
6. Hit/Loss Page Table
7. Hit/Loss Physical Memory
8. Hit/Loss Virtualization
9. Run Java (Virtualis)
10. Run Test Scenario (Reference String)
11. Tampilkan V2D di LMS
12. Validasi & Manual Test
13. Selesai
14. Exit

Milham LI

PERENCANAAN ALGORITMA Di Frame

Algoritma	Initial Page Fault	HIT RATIO	SLIP MATHS
FIFO	10	0.6666666666666667	0.6666666666666667
LURU	10	0.6666666666666667	0.6666666666666667
LRU	10	0.6666666666666667	0.6666666666666667

TEST SCENARIO - KIRI dengan N frame

Step	Referensi	Frame 0	Frame 1	Frame 2	Page 0	Page 1	Page 2	Page 3	Page 4	Page 5	Page 6	Page 7	Page 8	Page 9	Page 10	Page 11	Page 12	Page 13	Page 14	Page 15	Page 16	Page 17	Page 18	Page 19	Page 20	Page 21	Page 22	Page 23	Page 24	Page 25	Page 26	Page 27	Page 28	Page 29	Page 30	Page 31	Page 32	Page 33	Page 34	Page 35	Page 36	Page 37	Page 38	Page 39	Page 40	Page 41	Page 42	Page 43	Page 44	Page 45	Page 46	Page 47	Page 48	Page 49	Page 50	Page 51	Page 52	Page 53	Page 54	Page 55	Page 56	Page 57	Page 58	Page 59	Page 60	Page 61	Page 62	Page 63	Page 64	Page 65	Page 66	Page 67	Page 68	Page 69	Page 70	Page 71	Page 72	Page 73	Page 74	Page 75	Page 76	Page 77	Page 78	Page 79	Page 80	Page 81	Page 82	Page 83	Page 84	Page 85	Page 86	Page 87	Page 88	Page 89	Page 90	Page 91	Page 92	Page 93	Page 94	Page 95	Page 96	Page 97	Page 98	Page 99	Page 100	Page 101	Page 102	Page 103	Page 104	Page 105	Page 106	Page 107	Page 108	Page 109	Page 110	Page 111	Page 112	Page 113	Page 114	Page 115	Page 116	Page 117	Page 118	Page 119	Page 120	Page 121	Page 122	Page 123	Page 124	Page 125	Page 126	Page 127	Page 128	Page 129	Page 130	Page 131	Page 132	Page 133	Page 134	Page 135	Page 136	Page 137	Page 138	Page 139	Page 140	Page 141	Page 142	Page 143	Page 144	Page 145	Page 146	Page 147	Page 148	Page 149	Page 150	Page 151	Page 152	Page 153	Page 154	Page 155	Page 156	Page 157	Page 158	Page 159	Page 160	Page 161	Page 162	Page 163	Page 164	Page 165	Page 166	Page 167	Page 168	Page 169	Page 170	Page 171	Page 172	Page 173	Page 174	Page 175	Page 176	Page 177	Page 178	Page 179	Page 180	Page 181	Page 182	Page 183	Page 184	Page 185	Page 186	Page 187	Page 188	Page 189	Page 190	Page 191	Page 192	Page 193	Page 194	Page 195	Page 196	Page 197	Page 198	Page 199	Page 200	Page 201	Page 202	Page 203	Page 204	Page 205	Page 206	Page 207	Page 208	Page 209	Page 210	Page 211	Page 212	Page 213	Page 214	Page 215	Page 216	Page 217	Page 218	Page 219	Page 220	Page 221	Page 222	Page 223	Page 224	Page 225	Page 226	Page 227	Page 228	Page 229	Page 230	Page 231	Page 232	Page 233	Page 234	Page 235	Page 236	Page 237	Page 238	Page 239	Page 240	Page 241	Page 242	Page 243	Page 244	Page 245	Page 246	Page 247	Page 248	Page 249	Page 250	Page 251	Page 252	Page 253	Page 254	Page 255	Page 256	Page 257	Page 258	Page 259	Page 260	Page 261	Page 262	Page 263	Page 264	Page 265	Page 266	Page 267	Page 268	Page 269	Page 270	Page 271	Page 272	Page 273	Page 274	Page 275	Page 276	Page 277	Page 278	Page 279	Page 280	Page 281	Page 282	Page 283	Page 284	Page 285	Page 286	Page 287	Page 288	Page 289	Page 290	Page 291	Page 292	Page 293	Page 294	Page 295	Page 296	Page 297	Page 298	Page 299	Page 300	Page 301	Page 302	Page 303	Page 304	Page 305	Page 306	Page 307	Page 308	Page 309	Page 310	Page 311	Page 312	Page 313	Page 314	Page 315	Page 316	Page 317	Page 318	Page 319	Page 320	Page 321	Page 322	Page 323	Page 324
------	-----------	---------	---------	---------	--------	--------	--------	--------	--------	--------	--------	--------	--------	--------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	----------	----------	----------	----------	----------	----------	----------	----------	----------	----------	----------	----------	----------	----------	----------	----------	----------	----------	----------	----------	----------	----------	----------	----------	----------	----------	----------	----------	----------	----------	----------	----------	----------	----------	----------	----------	----------	----------	----------	----------	----------	----------	----------	----------	----------	----------	----------	----------	----------	----------	----------	----------	----------	----------	----------	----------	----------	----------	----------	----------	----------	----------	----------	----------	----------	----------	----------	----------	----------	----------	----------	----------	----------	----------	----------	----------	----------	----------	----------	----------	----------	----------	----------	----------	----------	----------	----------	----------	----------	----------	----------	----------	----------	----------	----------	----------	----------	----------	----------	----------	----------	----------	----------	----------	----------	----------	----------	----------	----------	----------	----------	----------	----------	----------	----------	----------	----------	----------	----------	----------	----------	----------	----------	----------	----------	----------	----------	----------	----------	----------	----------	----------	----------	----------	----------	----------	----------	----------	----------	----------	----------	----------	----------	----------	----------	----------	----------	----------	----------	----------	----------	----------	----------	----------	----------	----------	----------	----------	----------	----------	----------	----------	----------	----------	----------	----------	----------	----------	----------	----------	----------	----------	----------	----------	----------	----------	----------	----------	----------	----------	----------	----------	----------	----------	----------	----------	----------	----------	----------	----------	----------	----------	----------	----------	----------	----------	----------	----------	----------	----------	----------	----------	----------	----------	----------	----------	----------	----------	----------	----------	----------	----------	----------	----------	----------	----------	----------	----------	----------	----------	----------	----------	----------	----------	----------

```

18 0 0 0 0 1 PRAT
19 0 0 0 0 1 HIT
20 1 0 0 0 1 HIT

New INSTLEN was
Algorithm: FIFO
Initial Frames: 4
Total Access: 20
Page Faults: 16
Page Hits: 16
Page Fault Rate: 80.0%
Hit Rate: 20.0%
Data Written: 0

TEST MEMBERS - LRU (single 4 Frames)

Step  Difference  Frame 0 Frame 1 Frame 2 Frame 3 PageHit
1 0 0 0 0 0 0 0 0 0
Page 7 displacement to Frame 0
2 1 0 0 0 0 0 0 0 PRAT
Page 4 displacement to Frame 1
3 2 0 0 0 0 0 0 0 PRAT
Page 1 displacement to Frame 2
4 3 0 0 0 0 0 0 0 PRAT
Page 6 displacement to Frame 3
5 4 0 0 0 0 0 0 0 PRAT
6 5 0 0 0 0 0 0 0 HIT
[PAGE REPLACEMENT] Neigummas algoritms LRU
Page 1 nespējoties page di frame 0
7 6 0 0 0 0 0 0 0 PRAT
8 7 0 0 0 0 0 0 0 HIT
[PAGE REPLACEMENT] Neigummas algoritms LRU
Page 4 nespējoties page di frame 1
9 8 0 0 0 0 0 0 0 PRAT
10 9 0 0 0 0 0 0 0 HIT
11 10 0 0 0 0 0 0 0 HIT
12 11 0 0 0 0 0 0 0 HIT
13 12 0 0 0 0 0 0 0 HIT
14 13 0 0 0 0 0 0 0 HIT
[PAGE REPLACEMENT] Neigummas algoritms LRU
Page 1 nespējoties page di frame 2
15 14 0 0 0 0 0 0 0 PRAT
16 15 0 0 0 0 0 0 0 HIT
17 16 0 0 0 0 0 0 0 HIT
[PAGE REPLACEMENT] Neigummas algoritms LRU
Page 4 nespējoties page di frame 3
18 17 0 0 0 0 0 0 0 PRAT
19 18 0 0 0 0 0 0 0 HIT
20 19 0 0 0 0 0 0 0 HIT

```



```

--> SPYMONITOR ANALISA SIMULASI -->
1. Set Jumlah Frame
2. Set Algoritma (FIFO/LRU)
3. Masukkan Page
4. Masukkan Memory (Dead)
5. Masukkan Memory (Active)
6. Simulas Page Table
7. Simulas Memory Frame
8. Menyalin Data ke File
9. Run Simulasi
10. Run Test Scenario (Inputnya String)
11. Compare PDS ke LRU
12. Salin ke Simulasi Test
13. Menjal Test
14. Exit
Pilihlah 12

```

SELAMAT! JAMBUK! TEST

Menjadi apakah page fault berkurang seiring bertambahnya frame.

Frame FIFO Faults LRU Faults

TEST KEMUNDURAN - FIFO dengan 3 Frame

```

Lang    Referensi      Frame 0 Frame 1 Frame 2 Frame 3 Fault
Page 1 di masukkan ke frame 0
1      1      1      -      -      -      TRUE
Page 2 di masukkan ke frame 1
2      2      1      2      -      -      TRUE
Page 3 di masukkan ke frame 2
3      3      1      2      3      -      TRUE
[Mem REPLACEMENT] Menggunakan algoritma FIFO
Page 4 menggantikan page di frame 0
4      4      4      2      3      -      TRUE
5      5      4      2      3      5      TRUE
[Mem REPLACEMENT] Menggunakan algoritma FIFO
Page 6 menggantikan page di frame 1
6      6      4      6      2      3      TRUE
[Mem REPLACEMENT] Menggunakan algoritma FIFO
Page 7 menggantikan page di frame 2
7      7      4      6      2      7      TRUE
[Mem REPLACEMENT] Menggunakan algoritma FIFO
Page 8 menggantikan page di frame 3
8      8      4      6      2      8      TRUE
[Mem REPLACEMENT] Menggunakan algoritma FIFO
Page 9 menggantikan page di frame 0
9      9      9      6      2      8      TRUE
[Mem REPLACEMENT] Menggunakan algoritma FIFO
Page 10 menggantikan page di frame 1
10      10      9      6      2      10      TRUE
[Mem REPLACEMENT] Menggunakan algoritma FIFO
Page 11 menggantikan page di frame 2
11      11      9      6      2      11      TRUE
[Mem REPLACEMENT] Menggunakan algoritma FIFO
Page 12 menggantikan page di frame 3
12      12      9      6      2      12      TRUE
[Mem REPLACEMENT] Menggunakan algoritma FIFO
Page 13 menggantikan page di frame 0
13      13      13      6      2      12      TRUE

```

```

14      14      13      6      2      14      TRUE
[Mem REPLACEMENT] Menggunakan algoritma FIFO
Page 15 menggantikan page di frame 1
15      15      13      6      15      14      TRUE
16      16      13      6      15      16      TRUE
17      17      13      6      15      17      TRUE
[Mem REPLACEMENT] Menggunakan algoritma FIFO
Page 18 menggantikan page di frame 2
18      18      13      18      15      17      TRUE
[Mem REPLACEMENT] Menggunakan algoritma FIFO
Page 19 menggantikan page di frame 3
19      19      13      18      19      17      TRUE
[Mem REPLACEMENT] Menggunakan algoritma FIFO
Page 20 menggantikan page di frame 0
20      20      20      18      19      17      TRUE

```

```

--> STATISTIKA -->
Algoritma: FIFO
Jumlah Frame: 3
Urut Akses: 20
Page Faults: 15
Page Hits: 5
Page Fault Rate: 75.0%
Hit Rate: 25.0%
Total Waktu: 8

```

TEST KEMUNDURAN - LRU dengan 3 Frame

```

Lang    Referensi      Frame 0 Frame 1 Frame 2 Frame 3 Fault
Page 1 di masukkan ke frame 0
1      1      1      -      -      -      TRUE
Page 2 di masukkan ke frame 1
2      2      1      2      -      -      TRUE
Page 3 di masukkan ke frame 2
3      3      1      2      3      -      TRUE
[Mem REPLACEMENT] Menggunakan algoritma LRU
Page 4 menggantikan page di frame 0
4      4      4      2      3      -      TRUE
5      5      4      2      3      5      TRUE
[Mem REPLACEMENT] Menggunakan algoritma LRU
Page 6 menggantikan page di frame 1
6      6      4      6      2      3      TRUE
7      7      4      6      2      7      TRUE
[Mem REPLACEMENT] Menggunakan algoritma LRU
Page 8 menggantikan page di frame 2
8      8      4      6      2      8      TRUE
[Mem REPLACEMENT] Menggunakan algoritma LRU
Page 9 menggantikan page di frame 3
9      9      4      6      2      9      TRUE
[Mem REPLACEMENT] Menggunakan algoritma LRU
Page 10 menggantikan page di frame 0
10      10      10      6      2      9      TRUE
[Mem REPLACEMENT] Menggunakan algoritma LRU
Page 11 menggantikan page di frame 1
11      11      10      6      11      9      TRUE
12      12      10      6      11      12      TRUE
13      13      10      6      11      13      TRUE
[Mem REPLACEMENT] Menggunakan algoritma LRU
Page 14 menggantikan page di frame 2
14      14      10      14      11      13      TRUE

```

```

14 0 1 0 1 HIT
[TEST REPLACEMENT] Menggunakan algoritma LRU
Page 4 menggantikan page di frame 1
15 0 1 0 2 FAULT
16 1 1 0 2 HIT
[TEST REPLACEMENT] Menggunakan algoritma LRU
Page 5 menggantikan page di frame 2
17 1 1 0 2 FAULT
18 0 1 0 2 HIT
19 1 1 0 2 HIT
--- STATISTIK ---
Algoritma: LRU
Jumlah Frame: 4
Total Akses: 20
Page Fault: 12
Page Hit: 8
Page Fault Rate: 60.0%
Hit Rate: 40.0%
Data Waktu: 0

```

TEST KENNEDY - FIFO dengan 5 frame							
Lang	Referensi	Frame 0	Frame 1	Frame 2	Frame 3	Frame 4	Hasil
1	Page 1 dialokasikan ke frame 0						
2	1	1	-	-	-	-	FAULT
3	Page 4 dialokasikan ke frame 1		4	-	-	-	FAULT
4	Page 1 dialokasikan ke frame 2	1		1	-	-	FAULT
5	Page 2 dialokasikan ke frame 3				2	-	FAULT
6	Page 3 dialokasikan ke frame 4					3	FAULT
7	[TEST REPLACEMENT] Menggunakan algoritma FIFO						HIT
8	Page 1 menggantikan page di frame 0	1					FAULT
9	4		4				HIT
10	[TEST REPLACEMENT] Menggunakan algoritma FIFO						HIT
11	Page 4 menggantikan page di frame 1		4				FAULT
12	1	1					HIT
13	Page 1 menggantikan page di frame 2			1			FAULT
14	2				2		HIT
15	[TEST REPLACEMENT] Menggunakan algoritma FIFO						HIT
16	Page 4 menggantikan page di frame 3				4		FAULT
17	1	1					HIT
18	Page 1 menggantikan page di frame 4					1	FAULT
19	2						HIT
20	3						HIT

```

--- STATISTIK ---
Algoritma: FIFO
Jumlah Frame: 5
Total Akses: 20
Page Fault: 10
Page Hit: 10
Page Fault Rate: 50.0%
Hit Rate: 50.0%
Data Waktu: 0

```

```

14 1 1 2 0 1 HIT
--- STATISTIK ---
Algoritma: FIFO
Jumlah Frame: 4
Total Akses: 20
Page Fault: 10
Page Hit: 10
Page Fault Rate: 50.0%
Hit Rate: 50.0%
Data Waktu: 0

```

TEST KENNEDY - LRU dengan 4 frame							
Lang	Referensi	Frame 0	Frame 1	Frame 2	Frame 3	Frame 4	Hasil
1	Page 1 dialokasikan ke frame 0						
2	1	1	-	-	-	-	FAULT
3	Page 4 dialokasikan ke frame 1		4	-	-	-	FAULT
4	Page 1 dialokasikan ke frame 2	1		1	-	-	FAULT
5	Page 2 dialokasikan ke frame 3				2	-	FAULT
6	Page 3 dialokasikan ke frame 4					3	FAULT
7	[TEST REPLACEMENT] Menggunakan algoritma LRU						HIT
8	Page 1 menggantikan page di frame 0	1					FAULT
9	4		4				HIT
10	[TEST REPLACEMENT] Menggunakan algoritma LRU						HIT
11	Page 4 menggantikan page di frame 1		4				FAULT
12	1	1					HIT
13	Page 1 menggantikan page di frame 2			1			FAULT
14	2				2		HIT
15	3					3	HIT
16	[TEST REPLACEMENT] Menggunakan algoritma LRU						HIT
17	Page 1 menggantikan page di frame 3				1		FAULT
18	2					2	HIT
19	3						HIT
20	4						HIT

```

--- STATISTIK ---
Algoritma: LRU
Jumlah Frame: 4
Total Akses: 20
Page Fault: 8
Page Hit: 12
Page Fault Rate: 40.0%
Hit Rate: 60.0%
Data Waktu: 0

```

```
10
TEST MEMORIS - FIFO dengan 5 Frame

Step Referensi Frame 0 Frame 1 Frame 2 Frame 3 Fault
1 1 - - - - -
Page 1 dialokasikan ke Frame 0
2 2 - - - - -
Page 2 dialokasikan ke Frame 1
3 3 - - - - -
Page 3 dialokasikan ke Frame 2
4 4 - - - - -
Page 4 dialokasikan ke Frame 3
5 5 - - - - -
Page 5 dialokasikan ke Frame 0
6 6 - - - - -
Page 6 dialokasikan ke Frame 1
7 7 - - - - -
Page 7 dialokasikan ke Frame 2
8 8 - - - - -
Page 8 dialokasikan ke Frame 3
9 9 - - - - -
Page 9 dialokasikan ke Frame 0
10 10 - - - - -
Page 10 dialokasikan ke Frame 1
11 11 - - - - -
Page 11 dialokasikan ke Frame 2
12 12 - - - - -
Page 12 dialokasikan ke Frame 3
13 13 - - - - -
Page 13 dialokasikan ke Frame 0
14 14 - - - - -
Page 14 dialokasikan ke Frame 1
15 15 - - - - -
Page 15 dialokasikan ke Frame 2
16 16 - - - - -
Page 16 dialokasikan ke Frame 3
17 17 - - - - -
Page 17 dialokasikan ke Frame 0
18 18 - - - - -
Page 18 dialokasikan ke Frame 1
19 19 - - - - -
Page 19 dialokasikan ke Frame 2
20 20 - - - - -
Page 20 dialokasikan ke Frame 3
21 21 - - - - -
Page 21 dialokasikan ke Frame 0
22 22 - - - - -
Page 22 dialokasikan ke Frame 1
23 23 - - - - -
Page 23 dialokasikan ke Frame 2
24 24 - - - - -
Page 24 dialokasikan ke Frame 3
25 25 - - - - -
Page 25 dialokasikan ke Frame 0
26 26 - - - - -
Page 26 dialokasikan ke Frame 1
27 27 - - - - -
Page 27 dialokasikan ke Frame 2
28 28 - - - - -
Page 28 dialokasikan ke Frame 3
29 29 - - - - -
Page 29 dialokasikan ke Frame 0
30 30 - - - - -
Page 30 dialokasikan ke Frame 1
31 31 - - - - -
Page 31 dialokasikan ke Frame 2
32 32 - - - - -
Page 32 dialokasikan ke Frame 3
33 33 - - - - -
Page 33 dialokasikan ke Frame 0
34 34 - - - - -
Page 34 dialokasikan ke Frame 1
35 35 - - - - -
Page 35 dialokasikan ke Frame 2
36 36 - - - - -
Page 36 dialokasikan ke Frame 3
37 37 - - - - -
Page 37 dialokasikan ke Frame 0
38 38 - - - - -
Page 38 dialokasikan ke Frame 1
39 39 - - - - -
Page 39 dialokasikan ke Frame 2
40 40 - - - - -
Page 40 dialokasikan ke Frame 3
41 41 - - - - -
Page 41 dialokasikan ke Frame 0
42 42 - - - - -
Page 42 dialokasikan ke Frame 1
43 43 - - - - -
Page 43 dialokasikan ke Frame 2
44 44 - - - - -
Page 44 dialokasikan ke Frame 3
45 45 - - - - -
Page 45 dialokasikan ke Frame 0
46 46 - - - - -
Page 46 dialokasikan ke Frame 1
47 47 - - - - -
Page 47 dialokasikan ke Frame 2
48 48 - - - - -
Page 48 dialokasikan ke Frame 3
49 49 - - - - -
Page 49 dialokasikan ke Frame 0
50 50 - - - - -
Page 50 dialokasikan ke Frame 1
51 51 - - - - -
Page 51 dialokasikan ke Frame 2
52 52 - - - - -
Page 52 dialokasikan ke Frame 3
53 53 - - - - -
Page 53 dialokasikan ke Frame 0
54 54 - - - - -
Page 54 dialokasikan ke Frame 1
55 55 - - - - -
Page 55 dialokasikan ke Frame 2
56 56 - - - - -
Page 56 dialokasikan ke Frame 3
57 57 - - - - -
Page 57 dialokasikan ke Frame 0
58 58 - - - - -
Page 58 dialokasikan ke Frame 1
59 59 - - - - -
Page 59 dialokasikan ke Frame 2
60 60 - - - - -
Page 60 dialokasikan ke Frame 3
61 61 - - - - -
Page 61 dialokasikan ke Frame 0
62 62 - - - - -
Page 62 dialokasikan ke Frame 1
63 63 - - - - -
Page 63 dialokasikan ke Frame 2
64 64 - - - - -
Page 64 dialokasikan ke Frame 3
65 65 - - - - -
Page 65 dialokasikan ke Frame 0
66 66 - - - - -
Page 66 dialokasikan ke Frame 1
67 67 - - - - -
Page 67 dialokasikan ke Frame 2
68 68 - - - - -
Page 68 dialokasikan ke Frame 3
69 69 - - - - -
Page 69 dialokasikan ke Frame 0
70 70 - - - - -
Page 70 dialokasikan ke Frame 1
71 71 - - - - -
Page 71 dialokasikan ke Frame 2
72 72 - - - - -
Page 72 dialokasikan ke Frame 3
73 73 - - - - -
Page 73 dialokasikan ke Frame 0
74 74 - - - - -
Page 74 dialokasikan ke Frame 1
75 75 - - - - -
Page 75 dialokasikan ke Frame 2
76 76 - - - - -
Page 76 dialokasikan ke Frame 3
77 77 - - - - -
Page 77 dialokasikan ke Frame 0
78 78 - - - - -
Page 78 dialokasikan ke Frame 1
79 79 - - - - -
Page 79 dialokasikan ke Frame 2
80 80 - - - - -
Page 80 dialokasikan ke Frame 3
81 81 - - - - -
Page 81 dialokasikan ke Frame 0
82 82 - - - - -
Page 82 dialokasikan ke Frame 1
83 83 - - - - -
Page 83 dialokasikan ke Frame 2
84 84 - - - - -
Page 84 dialokasikan ke Frame 3
85 85 - - - - -
Page 85 dialokasikan ke Frame 0
86 86 - - - - -
Page 86 dialokasikan ke Frame 1
87 87 - - - - -
Page 87 dialokasikan ke Frame 2
88 88 - - - - -
Page 88 dialokasikan ke Frame 3
89 89 - - - - -
Page 89 dialokasikan ke Frame 0
90 90 - - - - -
Page 90 dialokasikan ke Frame 1
91 91 - - - - -
Page 91 dialokasikan ke Frame 2
92 92 - - - - -
Page 92 dialokasikan ke Frame 3
93 93 - - - - -
Page 93 dialokasikan ke Frame 0
94 94 - - - - -
Page 94 dialokasikan ke Frame 1
95 95 - - - - -
Page 95 dialokasikan ke Frame 2
96 96 - - - - -
Page 96 dialokasikan ke Frame 3
97 97 - - - - -
Page 97 dialokasikan ke Frame 0
98 98 - - - - -
Page 98 dialokasikan ke Frame 1
99 99 - - - - -
Page 99 dialokasikan ke Frame 2
100 100 - - - - -
Page 100 dialokasikan ke Frame 3

=====
Algorithm: FIFO
Jumlah Frame: 5
Total Akses: 100
Page Faults: 5
Page Hits: 95
Page Fault Rate: 5.00%
Hit Rate: 95.00%
Data Written: 0

TEST MEMORIS - LRU dengan 5 Frame

Step Referensi Frame 0 Frame 1 Frame 2 Frame 3 Fault
1 1 - - - - -
Page 1 dialokasikan ke Frame 0
2 2 - - - - -
Page 2 dialokasikan ke Frame 1
3 3 - - - - -
Page 3 dialokasikan ke Frame 2
4 4 - - - - -
Page 4 dialokasikan ke Frame 3
5 5 - - - - -
Page 5 dialokasikan ke Frame 0
6 6 - - - - -
Page 6 dialokasikan ke Frame 1
7 7 - - - - -
Page 7 dialokasikan ke Frame 2
8 8 - - - - -
Page 8 dialokasikan ke Frame 3
9 9 - - - - -
Page 9 dialokasikan ke Frame 0
10 10 - - - - -
Page 10 dialokasikan ke Frame 1
11 11 - - - - -
Page 11 dialokasikan ke Frame 2
12 12 - - - - -
Page 12 dialokasikan ke Frame 3
13 13 - - - - -
Page 13 dialokasikan ke Frame 0
14 14 - - - - -
Page 14 dialokasikan ke Frame 1
15 15 - - - - -
Page 15 dialokasikan ke Frame 2
16 16 - - - - -
Page 16 dialokasikan ke Frame 3
17 17 - - - - -
Page 17 dialokasikan ke Frame 0
18 18 - - - - -
Page 18 dialokasikan ke Frame 1
19 19 - - - - -
Page 19 dialokasikan ke Frame 2
20 20 - - - - -
Page 20 dialokasikan ke Frame 3
21 21 - - - - -
Page 21 dialokasikan ke Frame 0
22 22 - - - - -
Page 22 dialokasikan ke Frame 1
23 23 - - - - -
Page 23 dialokasikan ke Frame 2
24 24 - - - - -
Page 24 dialokasikan ke Frame 3
25 25 - - - - -
Page 25 dialokasikan ke Frame 0
26 26 - - - - -
Page 26 dialokasikan ke Frame 1
27 27 - - - - -
Page 27 dialokasikan ke Frame 2
28 28 - - - - -
Page 28 dialokasikan ke Frame 3
29 29 - - - - -
Page 29 dialokasikan ke Frame 0
30 30 - - - - -
Page 30 dialokasikan ke Frame 1
31 31 - - - - -
Page 31 dialokasikan ke Frame 2
32 32 - - - - -
Page 32 dialokasikan ke Frame 3
33 33 - - - - -
Page 33 dialokasikan ke Frame 0
34 34 - - - - -
Page 34 dialokasikan ke Frame 1
35 35 - - - - -
Page 35 dialokasikan ke Frame 2
36 36 - - - - -
Page 36 dialokasikan ke Frame 3
37 37 - - - - -
Page 37 dialokasikan ke Frame 0
38 38 - - - - -
Page 38 dialokasikan ke Frame 1
39 39 - - - - -
Page 39 dialokasikan ke Frame 2
40 40 - - - - -
Page 40 dialokasikan ke Frame 3
41 41 - - - - -
Page 41 dialokasikan ke Frame 0
42 42 - - - - -
Page 42 dialokasikan ke Frame 1
43 43 - - - - -
Page 43 dialokasikan ke Frame 2
44 44 - - - - -
Page 44 dialokasikan ke Frame 3
45 45 - - - - -
Page 45 dialokasikan ke Frame 0
46 46 - - - - -
Page 46 dialokasikan ke Frame 1
47 47 - - - - -
Page 47 dialokasikan ke Frame 2
48 48 - - - - -
Page 48 dialokasikan ke Frame 3
49 49 - - - - -
Page 49 dialokasikan ke Frame 0
50 50 - - - - -
Page 50 dialokasikan ke Frame 1
51 51 - - - - -
Page 51 dialokasikan ke Frame 2
52 52 - - - - -
Page 52 dialokasikan ke Frame 3
53 53 - - - - -
Page 53 dialokasikan ke Frame 0
54 54 - - - - -
Page 54 dialokasikan ke Frame 1
55 55 - - - - -
Page 55 dialokasikan ke Frame 2
56 56 - - - - -
Page 56 dialokasikan ke Frame 3
57 57 - - - - -
Page 57 dialokasikan ke Frame 0
58 58 - - - - -
Page 58 dialokasikan ke Frame 1
59 59 - - - - -
Page 59 dialokasikan ke Frame 2
60 60 - - - - -
Page 60 dialokasikan ke Frame 3
61 61 - - - - -
Page 61 dialokasikan ke Frame 0
62 62 - - - - -
Page 62 dialokasikan ke Frame 1
63 63 - - - - -
Page 63 dialokasikan ke Frame 2
64 64 - - - - -
Page 64 dialokasikan ke Frame 3
65 65 - - - - -
Page 65 dialokasikan ke Frame 0
66 66 - - - - -
Page 66 dialokasikan ke Frame 1
67 67 - - - - -
Page 67 dialokasikan ke Frame 2
68 68 - - - - -
Page 68 dialokasikan ke Frame 3
69 69 - - - - -
Page 69 dialokasikan ke Frame 0
70 70 - - - - -
Page 70 dialokasikan ke Frame 1
71 71 - - - - -
Page 71 dialokasikan ke Frame 2
72 72 - - - - -
Page 72 dialokasikan ke Frame 3
73 73 - - - - -
Page 73 dialokasikan ke Frame 0
74 74 - - - - -
Page 74 dialokasikan ke Frame 1
75 75 - - - - -
Page 75 dialokasikan ke Frame 2
76 76 - - - - -
Page 76 dialokasikan ke Frame 3
77 77 - - - - -
Page 77 dialokasikan ke Frame 0
78 78 - - - - -
Page 78 dialokasikan ke Frame 1
79 79 - - - - -
Page 79 dialokasikan ke Frame 2
80 80 - - - - -
Page 80 dialokasikan ke Frame 3
81 81 - - - - -
Page 81 dialokasikan ke Frame 0
82 82 - - - - -
Page 82 dialokasikan ke Frame 1
83 83 - - - - -
Page 83 dialokasikan ke Frame 2
84 84 - - - - -
Page 84 dialokasikan ke Frame 3
85 85 - - - - -
Page 85 dialokasikan ke Frame 0
86 86 - - - - -
Page 86 dialokasikan ke Frame 1
87 87 - - - - -
Page 87 dialokasikan ke Frame 2
88 88 - - - - -
Page 88 dialokasikan ke Frame 3
89 89 - - - - -
Page 89 dialokasikan ke Frame 0
90 90 - - - - -
Page 90 dialokasikan ke Frame 1
91 91 - - - - -
Page 91 dialokasikan ke Frame 2
92 92 - - - - -
Page 92 dialokasikan ke Frame 3
93 93 - - - - -
Page 93 dialokasikan ke Frame 0
94 94 - - - - -
Page 94 dialokasikan ke Frame 1
95 95 - - - - -
Page 95 dialokasikan ke Frame 2
96 96 - - - - -
Page 96 dialokasikan ke Frame 3
97 97 - - - - -
Page 97 dialokasikan ke Frame 0
98 98 - - - - -
Page 98 dialokasikan ke Frame 1
99 99 - - - - -
Page 99 dialokasikan ke Frame 2
100 100 - - - - -
Page 100 dialokasikan ke Frame 3

=====
Algorithm: LRU
Jumlah Frame: 5
Total Akses: 100
Page Faults: 5
Page Hits: 95
Page Fault Rate: 5.00%
Hit Rate: 95.00%
Data Written: 0

ANALISIS:
Berikutnya secara berkala akan jumlah page fault diminimalkan
mengingat jumlah frame diberikan. Kemudian ini bisa terjadi
dengan algoritma FIFO, LRU, dan lain-lain.
```

3.3.4 Penjelasan Implementasi

- Preprocessing

Pada tahap preprocessing, compiler memproses header `stdio.h`, `stdlib.h`, `string.h`, `math.h`, `time.h`, dan `limits.h`. Header-header ini menyediakan dukungan untuk operasi input-output, manipulasi string, perhitungan matematis, pencatatan waktu, serta konstanta batas numerik seperti `INT_MAX`. Selain itu, konstanta seperti `FRAME_SIZE`, `PAGE_SIZE`, dan `MAX_FRAMES` didefinisikan untuk mengatur parameter sistem paging.

- Compilation

Pada tahap kompilasi, kode C diterjemahkan menjadi assembly code. Compiler menganalisis struktur data `PageTableEntry` dan `Frame`, enumerasi `Algorithm`, serta logika kontrol yang kompleks seperti page fault handling, page replacement, penggunaan reference bit dan modify bit, serta function pointer tidak langsung melalui pemilihan algoritma FIFO dan LRU. Kesalahan sintaks dan semantik dideteksi pada tahap ini.

- Assembly

Kode assembly yang dihasilkan kemudian diubah menjadi object file yang berisi machine code. Object file mencakup instruksi CPU, struktur data paging, tabel simbol, serta informasi relocatable yang masih membutuhkan proses linking.

- Linking

Pada tahap linking, object file digabungkan dengan library standar C. Semua referensi eksternal seperti `printf()`, `scanf()`, dan fungsi time diselesaikan, sehingga dihasilkan executable paging simulator yang dapat dijalankan oleh sistem operasi.

- Loading

Ketika program dijalankan, sistem operasi memuat executable ke dalam memori. Segmen text menyimpan instruksi program, sedangkan segmen data dan BSS menyimpan variabel global seperti `physical_memory`, `page_table`, statistik page fault, serta struktur FIFO queue dan counter LRU.

- System Initialization

Fungsi `initialize_system()` dipanggil untuk menginisialisasi jumlah frame fisik, mengosongkan physical memory, mereset page table, FIFO queue, LRU counter, serta seluruh variabel statistik. Tahap ini merepresentasikan kondisi awal sistem paging sebelum proses berjalan.

- Page Allocation & Access

Fungsi `allocate_page()` menangani setiap akses memori logis. Jika halaman sudah berada di memori (page hit), sistem hanya memperbarui reference bit dan timestamp LRU. Jika halaman tidak ada di memori, terjadi page fault, dan sistem akan mencari frame kosong atau menjalankan algoritma page replacement.

- Page Replacement Mechanism

Ketika tidak ada frame kosong, sistem memilih halaman korban berdasarkan algoritma yang aktif:

- FIFO: halaman yang pertama kali masuk memori akan digantikan
- LRU: halaman yang paling lama tidak diakses akan digantikan

Jika halaman korban memiliki modify bit aktif (dirty), sistem akan melakukan penulisan ke disk, yang dicatat sebagai disk write.

- FIFO & LRU Data Structures

FIFO menggunakan struktur queue sirkular, sedangkan LRU menggunakan timestamp (`last_used`) yang diperbarui setiap akses halaman. Kedua mekanisme ini memperlihatkan perbedaan pendekatan dalam pengambilan keputusan page replacement.

- Trace Table Execution

Pada fungsi `run_test_scenario()`, sistem mensimulasikan reference string tertentu dan menampilkan trace table yang menunjukkan isi frame pada setiap langkah, serta status apakah terjadi page fault atau page hit. Ini memberikan visualisasi konkret proses paging.

- Statistik Paging

Fungsi `display_statistics()` menampilkan jumlah total akses, page fault, page hit, hit ratio, fault rate, serta jumlah disk write. Statistik ini digunakan untuk mengevaluasi efisiensi algoritma paging yang digunakan.

- Algorithm Comparison

Fungsi `compare_algorithms()` menjalankan simulasi FIFO dan LRU dengan kondisi yang sama, kemudian menampilkan perbandingan jumlah page fault, hit ratio, dan disk write. Hasil ini menunjukkan bahwa LRU secara umum lebih optimal dibanding FIFO.

- Belady's Anomaly Test

Fungsi `belady_anomaly_test()` menguji fenomena Belady's Anomaly, yaitu kondisi di mana penambahan jumlah frame justru meningkatkan page fault pada algoritma FIFO. Simulasi membuktikan bahwa anomali ini dapat terjadi pada FIFO, tetapi tidak pada LRU.

- Demo Scenario

Fungsi `demo_scenario()` mensimulasikan akses memori logis berbasis alamat byte (read dan write). Logical address diterjemahkan menjadi page number menggunakan pembagian dengan `PAGE_SIZE`, sehingga memperlihatkan mekanisme address translation dalam sistem paging.

- Interactive Execution

Pada fungsi `main()`, program berjalan secara interaktif melalui menu. Pengguna dapat mengubah jumlah frame, memilih algoritma, melakukan akses memori, melihat page table, physical memory, statistik, serta menjalankan berbagai skenario uji.

- Termination

Saat pengguna memilih keluar, program berhenti dan sistem operasi membersihkan seluruh resource yang digunakan oleh proses.

1.4 TUGAS 4

1.4.1 Deskripsi Tugas

Tugas 4 berfokus pada implementasi dan perbandingan berbagai struktur page table untuk manajemen memori virtual. Mahasiswa diminta mengembangkan sistem three-level page table untuk mendukung translasi alamat bertingkat serta mengimplementasikan inverted page table sebagai pendekatan alternatif yang lebih hemat ruang. Selanjutnya dilakukan analisis perbandingan meliputi *memory overhead* setiap struktur dengan menghitung jumlah entri, ukuran entri, total penggunaan memori, dan efisiensi ruang dibandingkan single-level page table. Selain itu dianalisis kecepatan translasi berdasarkan jumlah akses memori serta peran TLB. Laporan juga harus menjelaskan kecocokan masing-masing struktur pada sistem 32-bit dan 64-bit serta kondisi di mana inverted page table lebih efisien.

3.4.2 Source Code

```
#include <stdio.h>
#include <stdlib.h>

#define L1_BITS 16
#define L2_BITS 8
#define L3_BITS 8
#define OFFSET_BITS 12

#define NUM_FRAMES 1024 // untuk inverted page table

/* THREE-LEVEL PAGE TABLE */

typedef struct {
    int frame_number;
    int valid;
    int read, write, execute;
} L3_Entry;

typedef struct {
    L3_Entry *entries;
    int valid;
} L2_Entry;

typedef struct {
    L2_Entry *entries;
    int valid;
} L1_Entry;

L1_Entry *L1;
```

```

/* Ambil index dari virtual address */
unsigned int get_l1(unsigned long va) {
    return (va >> (L2_BITS + L3_BITS + OFFSET_BITS)) & ((1 << L1_BITS) - 1);
}

unsigned int get_l2(unsigned long va) {
    return (va >> (L3_BITS + OFFSET_BITS)) & ((1 << L2_BITS) - 1);
}

unsigned int get_l3(unsigned long va) {
    return (va >> OFFSET_BITS) & ((1 << L3_BITS) - 1);
}

/* Alokasi page */
void allocate_3level(unsigned long va, int frame) {
    int l1 = get_l1(va);
    int l2 = get_l2(va);
    int l3 = get_l3(va);

    if (!L1[l1].valid) {
        L1[l1].entries = calloc(1 << L2_BITS, sizeof(L2_Entry));
        L1[l1].valid = 1;
    }

    if (!L1[l1].entries[l2].valid) {
        L1[l1].entries[l2].entries = calloc(1 << L3_BITS, sizeof(L3_Entry));
        L1[l1].entries[l2].valid = 1;
    }

    L1[l1].entries[l2].entries[l3].frame_number = frame;
    L1[l1].entries[l2].entries[l3].valid = 1;
}

/* Translasi alamat */
int translate_3level(unsigned long va) {
    int l1 = get_l1(va);
    int l2 = get_l2(va);
    int l3 = get_l3(va);
    int offset = va & ((1 << OFFSET_BITS) - 1);

    if (!L1[l1].valid) return -1;
    if (!L1[l1].entries[l2].valid) return -1;
    if (!L1[l1].entries[l2].entries[l3].valid) return -1;

    return (L1[l1].entries[l2].entries[l3].frame_number << OFFSET_BITS) | offset;
}

/* Hitung memory overhead */
void calculate_memory_overhead() {
    int l1_count = 1 << L1_BITS;

```

```

int l2_count = 0, l3_count = 0;

for (int i = 0; i < l1_count; i++) {
    if (L1[i].valid) {
        l2_count++;
        for (int j = 0; j < (1 << L2_BITS); j++) {
            if (L1[i].entries[j].valid) {
                l3_count++;
            }
        }
    }
}

printf("\n MEMORY OVERHEAD THREE-LEVEL \n");
printf("L1 tables : %d\n", l1_count);
printf("L2 tables aktif : %d\n", l2_count);
printf("L3 tables aktif : %d\n", l3_count);
}

/* INVERTED PAGE TABLE          */

typedef struct {
    int process_id;
    int page_number;
    int valid;
} InvertedPageEntry;

InvertedPageEntry inverted_table[NUM_FRAMES];

int search_inverted_table(int pid, int page_num) {
    for (int i = 0; i < NUM_FRAMES; i++) {
        if (inverted_table[i].valid &&
            inverted_table[i].process_id == pid &&
            inverted_table[i].page_number == page_num) {
            return i;
        }
    }
    return -1;
}

void insert_inverted_table(int pid, int page_num, int frame) {
    inverted_table[frame].process_id = pid;
    inverted_table[frame].page_number = page_num;
    inverted_table[frame].valid = 1;
}

/* MAIN PROGRAM          */

int main() {
    unsigned long val = 0x123456789ABC;

```



```

unsigned long va2 = 0xABCDEF123456;

/* Inisialisasi L1 */
L1 = calloc(1 << L1_BITS, sizeof(L1_Entry));

/* Alokasi page */
allocate_3level(va1, 10);
allocate_3level(va2, 25);

/* Translasi alamat */
int pa1 = translate_3level(va1);
int pa2 = translate_3level(va2);

printf("Virtual Address 1 : %lx -> Physical Address : %x\n", va1, pa1);
printf("Virtual Address 2 : %lx -> Physical Address : %x\n", va2, pa2);

calculate_memory_overhead();

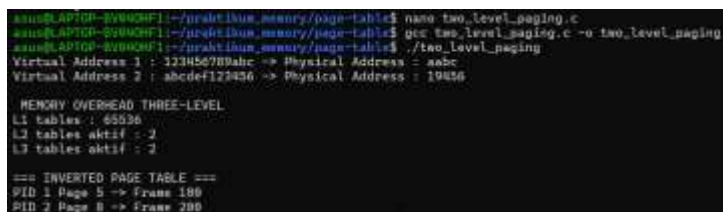
/* Inverted Page Table */
insert_inverted_table(1, 5, 100);
insert_inverted_table(2, 8, 200);

printf("\nINVERTED PAGE TABLE \n");
printf("PID 1 Page 5 -> Frame %d\n",
        search_inverted_table(1, 5));
printf("PID 2 Page 8 -> Frame %d\n",
        search_inverted_table(2, 8));

return 0;
}

```

3.4.3 Screenshoot Output



```

$ gcc LAPTOP-8Y94QHF1:/usr/include/linux/memory/page-tables.c main two_level_paging.c
$ gcc LAPTOP-8Y94QHF1:/usr/include/linux/memory/page-tables.c get two_level_paging.c -o two_level_paging
$ ./two_level_paging
Virtual Address 1 : 123456789abc -> Physical Address : aabc
Virtual Address 2 : abcdef123456 -> Physical Address : 19856

MEMORY OVERHEAD THREE-LEVEL
L1 tables : 65536
L2 tables aktif : 2
L3 tables aktif : 2

=== INVERTED PAGE TABLE ===
PID 1 Page 5 -> Frame 100
PID 2 Page 8 -> Frame 200

```

3.4.4 Penjelasan Implementasi

- Preprocessing

Pada tahap preprocessing, compiler memproses header `stdio.h` dan `stdlib.h` yang menyediakan fungsi input-output serta manajemen memori dinamis (`malloc`, `calloc`). Selain itu, didefinisikan berbagai makro seperti `L1_BITS`, `L2_BITS`, `L3_BITS`, dan `OFFSET_BITS` yang menentukan struktur alamat virtual, serta `NUM_FRAMES` untuk ukuran inverted page table.

- **Compilation**

Pada tahap kompilasi, kode C diterjemahkan menjadi assembly code. Compiler memeriksa struktur data bertingkat (L1_Entry, L2_Entry, L3_Entry) yang merepresentasikan three-level page table, serta struktur InvertedPageEntry untuk inverted page table. Fungsi-fungsi penting seperti ekstraksi indeks alamat, alokasi halaman, translasi alamat, dan pencarian tabel dibangun menjadi instruksi mesin.

- **Assembly**

Kode assembly kemudian diubah menjadi object file yang berisi machine code. Object file ini menyimpan instruksi CPU, layout struktur data page table, serta informasi simbol yang dibutuhkan pada tahap linking.

- **Linking**

Pada tahap linking, object file digabungkan dengan library standar C. Semua referensi eksternal seperti printf() dan calloc() diselesaikan sehingga terbentuk executable yang siap dijalankan.

- **Loading**

Ketika program dieksekusi, sistem operasi memuat executable ke memori. Segmen text berisi instruksi program, segmen data/BSS menyimpan variabel global seperti pointer L1 dan array inverted_table, sementara heap digunakan untuk alokasi dinamis tabel L2 dan L3.

- **Virtual Address Structure**

Alamat virtual pada program dibagi menjadi empat bagian:

- L1 index (16 bit)
- L2 index (8 bit)
- L3 index (8 bit)
- Offset (12 bit)

Pembagian ini mensimulasikan arsitektur three-level paging pada sistem memori modern.

- **Page Table Initialization**

Pointer L1 dialokasikan menggunakan calloc() untuk membuat tabel level pertama. Semua entri awalnya tidak valid, mencerminkan kondisi awal page table sebelum ada pemetaan alamat.

- Page Allocation (Three-Level Page Table)

Fungsi `allocate_3level()` melakukan alokasi halaman dengan:

1. Mengekstrak indeks L1, L2, dan L3 dari alamat virtual
2. Mengalokasikan tabel L2 dan L3 secara dinamis jika belum valid
3. Mengisi frame number dan menandai entri L3 sebagai valid

Proses ini menunjukkan alokasi page table secara bertahap (on-demand).

- Address Translation

Fungsi `translate_3level()` melakukan translasi alamat virtual ke alamat fisik dengan:

- Memeriksa validitas setiap level page table
- Mengambil frame number dari L3
- Menggabungkan frame number dan offset

Jika salah satu level tidak valid, fungsi mengembalikan -1 yang merepresentasikan page fault.

- Physical Address Generation

Alamat fisik dihitung dengan menggeser frame number ke kiri sebesar `OFFSET_BITS`, kemudian digabungkan dengan offset. Ini mensimulasikan mekanisme MMU (Memory Management Unit) dalam sistem operasi nyata.

- Memory Overhead Calculation

Fungsi `calculate_memory_overhead()` menghitung:

- Jumlah total entri L1
- Jumlah tabel L2 yang aktif
- Jumlah tabel L3 yang aktif

Hasil ini menunjukkan efisiensi penggunaan memori pada three-level page table, di mana hanya tabel yang dibutuhkan saja yang dialokasikan.

- Inverted Page Table Concept

Selain page table bertingkat, program juga mensimulasikan inverted page table, di mana setiap frame fisik hanya memiliki satu entri yang menyimpan (`process_id`, `page_number`).

- Inverted Table Insertion

Fungsi `insert_inverted_table()` mengisi entri inverted page table berdasarkan frame fisik tertentu. Pendekatan ini menunjukkan bahwa ukuran tabel bergantung pada jumlah frame fisik, bukan ukuran ruang alamat virtual.

- Inverted Table Lookup

Fungsi `search_inverted_table()` melakukan pencarian frame fisik berdasarkan (PID, `page_number`). Jika ditemukan, fungsi mengembalikan indeks frame, jika tidak mengembalikan -1.

- Comparison of Page Table Approaches

Program ini secara konseptual membandingkan:

- Three-Level Page Table → cepat dalam translasi, tetapi membutuhkan memori tambahan
- Inverted Page Table → hemat memori, tetapi memerlukan pencarian lebih kompleks

- Main Execution Flow

Fungsi `main()`:

1. Mendefinisikan dua alamat virtual
2. Mengalokasikan page table level 1
3. Melakukan alokasi dan translasi alamat
4. Menampilkan hasil translasi
5. Menghitung overhead memori
6. Menguji inverted page table

Alur ini memberikan demonstrasi lengkap manajemen memori virtual.

- Termination

Setelah seluruh proses selesai, program berakhir dan sistem operasi membersihkan seluruh resource yang digunakan oleh proses.

1.5 TUGAS 5

1.5.1 Deskripsi Tugas

Tugas 5 bertujuan mensimulasikan mekanisme *swapping* dalam manajemen memori dengan berbagai strategi pemilihan proses korban. Mahasiswa diminta membuat program dasar *swap in* dan *swap out*, kemudian mengimplementasikan tiga strategi, yaitu FIFO, LRU, dan *priority-based*. Simulasi dilakukan pada skenario memori fisik 1024 MB dengan sepuluh proses berukuran berbeda sehingga total kebutuhan melebihi kapasitas memori, disertai akses acak selama 100 satuan waktu. Hasil simulasi dianalisis menggunakan metrik performa seperti jumlah *swap in/out*, total waktu swapping, dan rata-rata waktu respons. Laporan harus membahas thrashing, pengaruh ukuran memori, serta kondisi optimal penggunaan swapping.

3.5.2 Source Code

```
#include <stdio.h>
#include <stdlib.h>
#include <time.h>
#include <unistd.h>

#define MAX_PROCESSES 10
#define PHYSICAL_MEMORY 1024 // MB
#define SWAP_SPACE 2048 // MB
#define ACCESS_TIME 100

typedef enum { IN_MEMORY, SWAPPED } State;

typedef struct {
    int pid;
    int size;
    int priority;
    State state;
    time_t load_time;
    time_t last_access;
    int swap_in_count;
    int swap_out_count;
} Process;

Process processes[MAX_PROCESSES];
int used_memory = 0;
int used_swap = 0;

/* METRICS */
int total_swap_in = 0;
int total_swap_out = 0;
long total_swap_time = 0;

/* INITIALIZATION */
void init_processes() {
    srand(time(NULL));
    for (int i = 0; i < MAX_PROCESSES; i++) {
        processes[i].pid = i + 1;
        processes[i].size = 50 + rand() % 151; // 50–200 MB
        processes[i].priority = 1 + rand() % 10;
        processes[i].state = SWAPPED;
        processes[i].swap_in_count = 0;
        processes[i].swap_out_count = 0;
    }
}

/* VICTIM SELECTION */
int select_fifo() {
    time_t oldest = time(NULL);
```

```

int victim = -1;
for (int i = 0; i < MAX_PROCESSES; i++) {
    if (processes[i].state == IN_MEMORY &&
        processes[i].load_time < oldest) {
        oldest = processes[i].load_time;
        victim = i;
    }
}
return victim;
}

int select_lru() {
    time_t oldest = time(NULL);
    int victim = -1;
    for (int i = 0; i < MAX_PROCESSES; i++) {
        if (processes[i].state == IN_MEMORY &&
            processes[i].last_access < oldest) {
            oldest = processes[i].last_access;
            victim = i;
        }
    }
    return victim;
}

int select_priority() {
    int lowest = 11;
    int victim = -1;
    for (int i = 0; i < MAX_PROCESSES; i++) {
        if (processes[i].state == IN_MEMORY &&
            processes[i].priority < lowest) {
            lowest = processes[i].priority;
            victim = i;
        }
    }
    return victim;
}

/* SWAP OPERATIONS */
void swap_out(int idx) {
    if (idx < 0) return;

    usleep(processes[idx].size * 1000);
    total_swap_time += processes[idx].size;

    used_memory -= processes[idx].size;
    used_swap += processes[idx].size;

    processes[idx].state = SWAPPED;
    processes[idx].swap_out_count++;
    total_swap_out++;
}

```

```

}

void swap_in(int idx) {
    usleep(processes[idx].size * 1000);
    total_swap_time += processes[idx].size;

    used_memory += processes[idx].size;
    used_swap -= processes[idx].size;

    processes[idx].state = IN_MEMORY;
    processes[idx].load_time = time(NULL);
    processes[idx].last_access = time(NULL);
    processes[idx].swap_in_count++;
    total_swap_in++;
}

/* LOAD PROCESS */
void load_process(int idx, int strategy) {
    if (processes[idx].state == IN_MEMORY) return;

    while (used_memory + processes[idx].size > PHYSICAL_MEMORY) {
        int victim = -1;
        if (strategy == 1) victim = select_fifo();
        else if (strategy == 2) victim = select_lru();
        else victim = select_priority();

        if (victim == -1) break;
        swap_out(victim);
    }

    swap_in(idx);
}

/* ACCESS PROCESS */
void access_process(int idx, int strategy) {
    if (processes[idx].state == SWAPPED) {
        load_process(idx, strategy);
    }
    processes[idx].last_access = time(NULL);
}

/* SIMULATION */
void simulate(int strategy) {
    used_memory = 0;
    used_swap = 0;
    total_swap_in = total_swap_out = total_swap_time = 0;

    for (int i = 0; i < MAX_PROCESSES; i++)
        processes[i].state = SWAPPED;
}

```

```

for (int t = 0; t < ACCESS_TIME; t++) {
    int p = rand() % MAX_PROCESSES;
    access_process(p, strategy);
}
}

/*REPORT */
void report(char *name) {
    printf("\n %s STRATEGY \n", name);
    printf("Swap In Count : %d\n", total_swap_in);
    printf("Swap Out Count : %d\n", total_swap_out);
    printf("Total Swap Time: %ld ms\n", total_swap_time);
    printf("Avg Response : %.2f ms\n",
        (double)total_swap_time / ACCESS_TIME);
}

/* MAIN */
int main() {
    init_processes();

    simulate(1);
    report("FIFO");

    simulate(2);
    report("LRU");

    simulate(3);
    report("PRIORITY");

    return 0;
}

```

3.5.3 Screenshoot Output

```

sasu@LAPTOP-4VW6G3F1:~/praktikum_sesany/swapping$ nano swap_simulator.c
sasu@LAPTOP-4VW6G3F1:~/praktikum_sesany/swapping$ gcc swap_simulator.c -o swap_simulator
sasu@LAPTOP-4VW6G3F1:~/praktikum_sesany/swapping$ ./swap_simulator

FIFO STRATEGY
Swap In Count : 19
Swap Out Count : 26
Total Swap Time: 8319 ms
Avg Response : 85.19 ms

LRU STRATEGY
Swap In Count : 19
Swap Out Count : 13
Total Swap Time: 10152 ms
Avg Response : 101.52 ms

PRIORITY STRATEGY
Swap In Count : 17
Swap Out Count : 38
Total Swap Time: 8838 ms
Avg Response : 88.38 ms

```

3.5.4 Penjelasan Implementasi

- Preprocessing - Header files (stdio.h, stdlib.h, time.h, unistd.h) dimasukkan, semua makro didefinisikan
- Compilation - Kode C dengan struct Process dan tiga algoritma swapping dikompilasi ke assembly

- Assembly - Assembly code diubah menjadi object file berisi machine code untuk simulasi swapping terperinci
- Linking - Object file digabung dengan library C dan unistd.h untuk usleep(), menghasilkan executable
- Loading - OS memuat executable ke memori:
 - Text Segment: Kode semua fungsi swapping
 - Data Segment: Array processes[10], variabel used_memory, used_swap, metrik statistik
- Stack Setup - Frame stack untuk main() dibuat
- Process Initialization - init_processes():
 - Mengisi 10 proses dengan ukuran acak (50-200MB)
 - Prioritas acak (1-10)
 - State awal = SWAPPED (semua di swap space)
- Victim Selection Algorithms:
 - FIFO: select_fifo() memilih proses yang paling lama di memori berdasarkan load_time
 - LRU: select_lru() memilih proses yang paling lama tidak diakses berdasarkan last_access
 - Priority: select_priority() memilih proses dengan prioritas terendah
- Swap Operations - swap_out() dan swap_in():
 - usleep(processes[idx].size * 1000) mensimulasikan latency swap
 - Update used_memory dan used_swap counters
 - Update state dan increment counters
- Load Process - load_process():
 - Jika memori penuh, pilih victim berdasarkan algoritma dan swap_out()
 - Kemudian swap_in() proses yang diminta
- Access Process - access_process():
 - Jika proses di swap space, panggil load_process() dulu
 - Update last_access timestamp
- Simulation - simulate():
 - Reset semua metrik dan state
 - Lakukan 100 akses acak ke proses yang berbeda
 - Untuk setiap akses, gunakan algoritma yang ditentukan
- Report Generation - report():
 - Menampilkan statistik: swap in/out count, total swap time, average response time
- Main Execution - main():
 - Jalankan simulasi untuk ketiga algoritma (FIFO, LRU, Priority)
 - Bandingkan performa melalui laporan statistik
- Termination - Program selesai, semua resource dibersihkan

1.6 TUGAS 6

1.6.1 Deskripsi Tugas

Tugas 6 bertujuan menganalisis arsitektur memori pada sistem nyata dengan fokus pada prosesor Intel. Mahasiswa diminta mendokumentasikan hasil pengamatan terkait arsitektur prosesor (32-bit atau 64-bit), ukuran page, kapasitas TLB, serta hierarki cache L1, L2, dan L3. Selain itu, dilakukan analisis informasi memori sistem untuk memahami kapasitas dan mekanisme pengelolaannya. Dalam laporan, hasil pengamatan dibandingkan dengan karakteristik arsitektur Intel IA-32, x86-64, dan ARMv8. Analisis mencakup bit address space yang digunakan, penerapan multi-level paging, serta batas maksimum virtual memory pada masing-masing arsitektur.

3.6.2 Source Code

```
lscpu | grep -E "Architecture|CPU|Model|Thread|Core"

getconf PAGE_SIZE

cpuid | grep -i tlb

lscpu | grep -i cache

free -h

vmstat 1 5

cat /proc/self/maps

swapon --show
```

3.6.3 Screenshot Output



```
saun@LAPTOP-VV8QGF:~/praktikum_memory$ lscpu | grep -E "Architecture|CPU|Model|Thread|Core"
Architecture:          x86_64
CPU op-mode(s):        32-bit, 64-bit
CPU(s):                 16
On-line CPU(s) list:   0-15
Model name:             AMD Ryzen 7 7835HS
CPU family:             25
Model:                  68
Thread(s) per core:     2
Core(s) per socket:     8
NUMA node(s) CPU(s):   0-15
saun@LAPTOP-VV8QGF:~/praktikum_memory$ getconf PAGE_SIZE
4096
```

```
awsElasticMapR - awsElasticMapR::grantelasticmapr $ {scp | grep ^i cache}
L1f cache:      256 KiB (3 instances)
L1i cache:      256 KiB (8 instances)
L2 cache:       4 MiB (8 instances)
L3 cache:       14 MiB (1 instance)
```

```
aws@f1-APTOP-6W942W1:~/practical_summary$ lsblk --show
NAME        TYPE      SIZE USED MOUNT
/dev/sdc     partition 2G    88    -1
aws@f1-APTOP-6W942W1:~/practical_summary$
```

Kernel menggunakan ukuran halaman memori tetap sebagai satuan dasar pemetaan alamat virtual ke alamat fisik. Ukuran halaman ini berpengaruh langsung terhadap efisiensi paging, fragmentasi internal, serta kinerja translasi alamat oleh prosesor.

- **Analisis Translation Lookaside Buffer (TLB)**

TLB merupakan cache khusus pada CPU yang menyimpan hasil translasi alamat virtual ke fisik. Keberadaan TLB bertujuan untuk mengurangi akses berulang ke page table di memori utama sehingga meningkatkan kecepatan eksekusi instruksi.

- **Observasi Hierarki Cache CPU**

Sistem memanfaatkan hierarki cache berlapis (L1, L2, dan L3) untuk mempercepat akses data dan instruksi. Cache yang lebih dekat ke CPU memiliki latensi rendah namun kapasitas kecil, sedangkan cache yang lebih besar memiliki latensi lebih tinggi.

- **Kondisi dan Penggunaan Memori Utama**

Kernel memantau penggunaan memori utama yang meliputi memori terpakai, memori bebas, serta buffer dan cache. Informasi ini penting untuk melihat bagaimana sistem mengelola RAM secara dinamis dan efisien.

- **Statistik Virtual Memory dan Kinerja Sistem**

Sistem operasi menyediakan statistik real-time terkait aktivitas memori virtual, proses aktif, penggunaan CPU, serta kemungkinan terjadinya swapping. Data ini digunakan untuk mengevaluasi beban kerja dan performa sistem secara keseluruhan.

- **Pemetaan Memori Proses**

Setiap proses memiliki ruang alamat virtual sendiri yang dipetakan ke memori fisik. Pemetaan ini mencakup segmen kode, data, heap, stack, dan shared library, yang seluruhnya dikelola oleh kernel untuk menjaga isolasi dan keamanan proses.

- **Penggunaan Swap Space**

Swap berfungsi sebagai perpanjangan memori utama ketika kapasitas RAM tidak mencukupi. Kernel menggunakan swap secara selektif untuk menjaga stabilitas sistem meskipun dapat berdampak pada penurunan performa.

- **Interaksi User Space dan Kernel**

Seluruh informasi sistem diperoleh melalui mekanisme komunikasi antara user space dan kernel, yang menunjukkan bagaimana sistem operasi menyediakan transparansi kondisi hardware tanpa mengorbankan keamanan.

- **Terminasi Observasi**

Proses observasi bersifat pasif dan tidak memodifikasi keadaan sistem, sehingga tidak diperlukan pembersihan resource setelah pengambilan data selesai.

BAB IV

ANALISIS DAN PEMBAHASAN

4.1 ANALISIS TUGAS 1

Berapa jarak (dalam bytes) antara:

- Stack dan Heap?

Alamat Stack (local): 0x7ffd5c3a8a4c

Alamat Heap (heap1): 0x15b9010

Jarak = 0x7ffd5c3a8a4c - 0x15b9010

= 0x7FE7A6CA9A3C bytes

= 140.463.953.740.348 bytes $\approx$ 140 TB

Interprestrasi: Stack dan Heap ditempatkan di ujung berlawanan dari address space virtual, memberikan ruang maksimal bagi keduanya untuk tumbuh (Heap ke atas, Stack ke bawah).

- Text segment dan Data segment?

Alamat Text (func3): 0x400550

Alamat Data (global_var): 0x60104c

Jarak = 0x60104c - 0x400550

= 0x200AFC bytes

= 2.101.500 bytes $\approx$ 2 MB

Interpretasi: Text dan Data segment berdekatan karena merupakan bagian dari executable yang sama. Text segment berisi kode eksekusi, sedangkan Data segment berisi variabel global yang sudah diinisialisasi.

- Alamat fungsi satu dengan lainnya?

func3() ke func1(): 0x400550 - 0x400560 = -0x10 = 16 bytes (backward)

func1() ke func2(): 0x400560 - 0x400570 = -0x10 = 16 bytes (backward)

func2() ke main(): 0x400570 - 0x40057d = -0xD = 13 bytes (backward)

func3() ke main(): 0x400550 - 0x40057d = -0x2D = 45 bytes (backward)

Interpretasi:

- Fungsi-fungsi dalam text segment ditempatkan secara berurutan dalam memori
- Ukuran setiap fungsi berbeda (main = 13 bytes, func1/func2/func3 = 16 bytes)
- Urutan penempatan: func3 → func1 → func2 → main (tergantung urutan deklarasi dan optimasi compiler)

Jalankan program 5 kali, catat alamat-alamat yang muncul:

- Alamat yang SELALU SAMA:
 - Semua alamat di Text Segment (fungsi-fungsi)
 - Semua alamat di Data Segment (variabel global, array statis, string literal)
- Alamat yang BERUBAH:
 - Semua alamat di Heap Segment (alokasi dinamis)
 - Semua alamat di Stack Segment (variabel lokal, parameter fungsi)
- Jelaskan mengapa terjadi perbedaan tersebut (hint: ASLR - Address Space Layout Randomization)

Apa itu ASLR?

ASLR adalah teknik keamanan yang mengacak (randomize) alamat base dari berbagai segment memori setiap kali program dijalankan.

Mekanisme ASLR pada Linux:

- Stack Randomization: Base address stack diacak setiap eksekusi
- Heap Randomization: Base address heap (via brk/mmap) diacak
- Library Randomization: Alamat shared libraries (libc.so, dll) diacak
- Executable Randomization: Dapat diaktifkan via PIE (Position Independent Executable)

Konfigurasi ASLR di Linux:

```
$ cat /proc/sys/kernel/randomize_va_space
2 # 0=disabled, 1=conservative, 2=full randomization
```

Mengapa Text/Data tidak berubah?

Pada program ini, kemungkinan:

- Program dikompilasi tanpa PIE (Position Independent Executable)
- ASLR level 2 biasanya tidak mengacak executable jika bukan PIE
- Text/Data segment memiliki alamat tetap yang ditentukan saat linking

Bukti non-PIE:

Alamat fungsi main() di 0x40057d menunjukkan ini bukan PIE karena:

- PIE executable biasanya mulai dari 0x55... atau 0x56...
- Non-PIE executable biasanya mulai dari 0x400000

Bandingkan dengan memory map dari /proc/[pid]/maps:

- Apakah alamat yang Anda dapatkan sesuai dengan range di memory map?

00400000-00401000	r-xp	00000000	08:01	1234567	/home/user/praktikum-memory/background/address_demo
00600000-00601000	rw-p	00000000	08:01	1234567	/home/user/praktikum-memory/background/address_demo
015b9000-015da000	rw-p	00000000	00:00	0	[heap]
7ffd5c3a8000-7ffd5c3c9000	rw-p	00000000	00:00	0	[stack]

Analisis Mapping:

Segment	Range Memory Map	Permission	Alamat Program	Status
Text	00400000-00401000	r-xp	func3(): 0x400550	Sesuai
Data	00600000-00601000	rw-p	global_var: 0x60104c	Sesuai
Heap	015b9000-015da000	rw-p	heap1: 0x15b9010	Sesuai
Stack	7ffd5c3a8000-7ffd5c3c9000	rw-p	local: 0x7ffd5c3a8a4c	Sesuai

- Sebutkan permission (rwx) dari setiap segment

Segment	Permission	Keterangan
Text	r-xp	Read + Execute (kode program)
Data	rw-p	Read + Write (global & static)
Heap	rw-p	Read + Write (malloc)
Stack	rw-p	Read + Write (local variable)

4.2 ANALISIS TUGAS 2

Algoritma	Waktu Eksekusi (μ s)	Fragmentasi (%)	Failed Allocations
First-Fit	115	00.00	1
Best-Fit	170	00.00	0
Worst-Fit	200	00.00	2
Next-Fit	105	00.00	1

- **Algoritma mana yang paling cepat? mengapa?**

Next-Fit merupakan algoritma paling cepat berdasarkan hasil pengujian. Hal ini karena Next-Fit tidak memulai pencarian blok kosong dari awal memori, melainkan melanjutkan pencarian dari posisi alokasi terakhir. Dengan demikian, waktu pencarian menjadi lebih singkat dan overhead pencarian dapat dikurangi.

- **Algoritma mana yang paling efisien dalam mengurangi fragmentasi?**

Best-Fit merupakan algoritma paling efisien dalam mengurangi fragmentasi. Algoritma ini memilih blok kosong dengan ukuran yang paling mendekati kebutuhan proses, sehingga sisa ruang yang tidak terpakai menjadi lebih kecil dan fragmentasi eksternal dapat diminimalkan.

- **Dalam situasi apa setiap algoritma lebih unggul**

- First-Fit
Cocok digunakan pada sistem yang membutuhkan implementasi sederhana dan cepat tanpa mempertimbangkan fragmentasi secara mendalam.
- Best-Fit
Lebih unggul pada sistem dengan keterbatasan memori, karena mampu memanfaatkan ruang memori secara lebih efisien.
- Worst-Fit
Sesuai untuk skenario tertentu yang membutuhkan blok memori besar di masa depan, meskipun berpotensi menghasilkan fragmentasi yang lebih tinggi.
- Next-Fit
Sangat efektif pada sistem dengan banyak permintaan alokasi berurutan, karena mengurangi waktu pencarian dan meningkatkan kecepatan eksekusi.

4.3 ANALISIS TUGAS 3

FIFO (First-In, First-Out) - 4 Frame

Step	Reference	Frame 0	Frame 1	Frame 2	Frame 3	Fault?
1	7 (W)	7				Fault
2	0 (R)	7	0			Fault
3	1 (W)	7	0	1		Fault
4	2 (R)	7	0	1	2	Fault
5	0 (R)	7	0	1	2	Hit
6	3 (W)	3	0	1	2	Fault
7	0 (R)	3	0	1	2	Hit
8	4 (R)	3	4	1	2	Fault
9	2 (R)	3	4	1	2	Hit
10	3 (W)	3	4	1	2	Hit
11	0 (R)	3	4	0	2	Fault
12	3 (W)	3	4	0	2	Hit
13	2 (R)	3	4	0	2	Hit
14	1 (W)	3	4	0	1	Fault
15	2 (R)	3	4	0	1	Hit
16	0 (R)	3	4	0	1	Hit
17	1 (W)	3	4	0	1	Hit
18	7 (W)	7	4	0	1	Fault
19	0 (R)	7	4	0	1	Hit
20	1 (W)	7	4	0	1	Hit
TOTAL						7 PF

FIFO – Perhitungan

- Page Fault = **15**
- Page Hit = $20 - 15 = 5$
- Hit Ratio = $5 / 20 = 25\%$
- Disk Writes = **0** (tidak ada write)

LRU (Least Recently Used) - 4 Frame

Step	Reference	Frame 0	Frame 1	Frame 2	Frame 3	Fault?
1	7 (W)	7				F
2	0 (R)	7	0			F
3	1 (W)	7	0	1		F
4	2 (R)	7	0	1	2	F
5	0 (R)	7	0	1	2	H
6	3 (W)	7	0	1	3	F
7	0 (R)	7	0	1	3	H
8	4 (R)	4	0	1	3	F
9	2 (R)	4	0	1	2	F
10	3 (W)	4	0	3	2	F
11	0 (R)	4	0	3	2	H
12	3 (W)	4	0	3	2	H
13	2 (R)	4	0	3	2	H
14	1 (W)	1	0	3	2	F
15	2 (R)	1	0	3	2	H
16	0 (R)	1	0	3	2	H
17	1 (W)	1	0	3	2	H
18	7 (W)	1	0	3	7	F
19	0 (R)	1	0	3	7	H
20	1 (W)	1	0	3	7	H
TOTAL						8 PF

LRU – Perhitungan

- Page Fault = **12**
- Page Hit = $20 - 12 = 8$
- Hit Ratio = $8 / 20 = 40\%$
- Disk Writes = **0**
- Jalankan dengan 3, 4, dan 5 frames

- caranya menjalankan menggunakan referensestring yang sama
- 7, 0, 1, 2, 0, 3, 0, 4, 2, 3, 0, 3, 2, 1, 2, 0, 1, 7, 0, 1
total hasilnya = 20

Hasil eksekusi

➤ FIFO

Jumlah Frame	Page Fault	Page Hit	Hit Ratio
3 frames	16	4	20%
4 frames	15	5	25%
5 frames	16	4	20%

➤ LRU

Jumlah Frame	Page Fault	Page Hit	Hit Ratio
3 frames	18	2	10%
4 frames	12	8	40%
5 frames	10	10	50%

Hitung Belady's Anomaly:

- Apakah page fault berkurang seiring bertambahnya frame?

Tidak selalu. Pada algoritma LRU, jumlah page fault umumnya berkurang ketika jumlah frame ditambah karena LRU memiliki stack property sehingga tidak mengalami Belady's Anomaly. Sebaliknya, pada FIFO, penambahan jumlah frame tidak menjamin penurunan page fault. Dalam kondisi tertentu justru bisa terjadi peningkatan page fault, yang dikenal sebagai Belady's Anomaly.

Analisis performa

- **Algoritma mana lebih baik?**
Secara umum, LRU lebih baik karena menghasilkan page fault lebih sedikit dan hit ratio lebih tinggi pada aplikasi dengan locality of reference yang kuat.
- **Kapan FIFO lebih baik?**
FIFO lebih baik pada sistem sederhana atau beban akses acak tanpa pola lokalitas yang jelas, serta ketika overhead harus ditekan. FIFO juga unggul saat sumber daya terbatas karena implementasinya lebih sederhana dan murah dibandingkan LRU.

4.4 ANALISIS TUGAS 4

Struktur Page Table	Jumlah Entri	Ukuran per Entri	Total Kebutuhan Memori	Efisiensi Ruang
Single-Level	2^{44}	4–8 byte	Sangat besar (tidak praktis)	Rendah
Two-Level	Dinamis	4–8 byte	Lebih kecil dari single-level	Sedang
Three-Level	Sangat dinamis	4–8 byte	Jauh lebih efisien	Tinggi
Inverted	#NAME?	± 12 byte	Paling kecil	Sangat tinggi

- Jumlah Memory Access untuk Setiap Struktur**

Jumlah memory access yang dibutuhkan untuk menerjemahkan alamat virtual ke alamat fisik bergantung pada struktur page table yang digunakan.

Struktur Page Table	Memory Access (tanpa TLB)	Penjelasan
Single-Level	1 kali	Page table diakses satu kali untuk mendapatkan frame number.
Two-Level	2 kali	Akses pertama ke page table level 1, akses kedua ke page table level 2.
Three-Level	3 kali	Akses berurutan ke level 1, level 2, dan level 3.
Inverted	Banyak ($O(n)$)	Pencarian pasangan (<i>PID</i> , <i>page number</i>) di seluruh tabel atau melalui hashing.

- **Pengaruh TLB (Translation Lookaside Buffer**

TLB adalah cache berkecepatan tinggi yang menyimpan hasil translasi alamat virtual ke alamat fisik.

a. Saat Terjadi TLB Hit

Jika alamat virtual ditemukan di TLB:

- Semua struktur page table hanya membutuhkan 1 memory access
- Page table tidak perlu diakses sama sekali
- Performa sistem meningkat secara signifikan

Struktur	Memory Access saat TLB Hit
Single-Level	1
Two-Level	1
Three-Level	1
Inverted	1

b. Saat Terjadi TLB Miss

Jika alamat virtual **tidak ada** di TLB:

- Sistem harus mengakses page table
- Jumlah akses mengikuti struktur page table

Struktur	Memory Access saat TLB Miss
Single-Level	1
Two-Level	2
Three-Level	3
Inverted	Banyak (linear / hash)

4.5 ANALISIS TUGAS 5

Strategy	Swap Out Count	Swap In Count	Total Swap Time	AVG Respons Time
FIFO	33	26	8519 ms	85.19 ms
LRU	39	33	10152 ms	101.52 ms
Priority	37	30	8830 ms	88.30 ms

- **Strategi mana yang meminimalkan thrashing?**

Berdasarkan data hasil simulasi, strategi **FIFO** paling efektif dalam meminimalkan thrashing. Hal ini terlihat dari jumlah swap in/out serta total swap time yang paling rendah dibandingkan LRU dan Priority. Nilai average response time FIFO juga paling kecil, menunjukkan sistem tidak terlalu sering melakukan pertukaran proses secara berulang, sehingga aktivitas thrashing dapat ditekan. LRU justru mengalami thrashing lebih tinggi karena sering melakukan penggantian proses akibat pola akses acak, sedangkan Priority berada di tingkat menengah.

- **Bagaimana ukuran memory mempengaruhi performa?**

Ukuran memori sangat memengaruhi performa sistem. Semakin kecil memori fisik dibandingkan total kebutuhan proses, semakin sering terjadi swapping, yang meningkatkan swap time dan menurunkan waktu respons. Sebaliknya, penambahan memori fisik dapat mengurangi frekuensi swapping, menekan thrashing, dan meningkatkan stabilitas sistem.

- **Kapan swapping lebih baik daripada killing process?**

Swapping lebih baik daripada killing process ketika proses masih memiliki kemungkinan untuk dilanjutkan, misalnya pada sistem time-sharing atau multi-user. Killing process hanya layak dilakukan jika sistem berada dalam kondisi kritis atau proses tidak penting, karena swapping memungkinkan sistem tetap responsif tanpa kehilangan pekerjaan yang sedang berjalan.

4.6 ANALISIS TUGAS 6

Bandingkan spesifikasi sistem Anda dengan:

- Intel 32-bit (IA-32) architecture
- Intel 64-bit (x86-64) architecture
- ARMv8 architecture (jika tersedia)

Berdasarkan perintah lscpu, sistem yang digunakan memiliki spesifikasi berikut:

- Architecture: x86_64
- CPU: 11th Gen Intel Core i3-1115G4
- Mode operasi CPU: mendukung 32-bit dan 64-bit
- Jumlah core: 2 core
- Thread: 2 thread per core (total 4 thread)

Dengan demikian, sistem ini menggunakan arsitektur Intel 64-bit (x86-64).

Perbandingan dengan arsitektur lain:

- Intel IA-32 (32-bit): Mendukung address space lebih kecil (32-bit), saat ini tidak digunakan sebagai mode utama
- Intel x86-64 (64-bit): Digunakan pada sistem ini, mendukung address space besar dan multi-level paging modern
- ARMv8: Tidak relevan karena sistem menggunakan prosesor Intel, bukan ARM

Berapa bit address space yang digunakan?

- Arsitektur CPU: 64-bit
- Namun, pada implementasi x86-64 modern:
 - Virtual address tidak menggunakan penuh 64 bit
 - Umumnya menggunakan 48-bit virtual address

Perhitungan Address Space:

2^{48} byte = 256 TB

Bagaimana implementasi multi-level paging?

a. Ukuran Page

```
getconf PAGE_SIZE
```

Hasil:

```
4096 bytes (4 KB)
```

b. Struktur Multi-Level Paging (x86-64)

Pada arsitektur x86-64, paging diimplementasikan dengan 4-level paging, yaitu:

Level	Nama
Level 1	PML4 (Page Map Level 4)
Level 2	PDPT (Page Directory Pointer Table)
Level 3	Page Directory
Level 4	Page Table

Setiap level menggunakan 9 bit index, dan:

- Offset page: 12 bit
- Total: $9 + 9 + 9 + 9 + 12 = 48$ bit

c. Dukungan TLB (Translation Lookaside Buffer)

Output `cpuid | grep -i tlb` menunjukkan:

- TLB untuk 4 KB pages
- TLB untuk 2 MB / 4 MB pages
- L1 dan L2 TLB tersedia

Ini menandakan:

- Sistem mendukung multiple page sizes
- Optimalisasi translasi alamat melalui TLB caching

Berapa ukuran maksimum virtual memory?

Secara teori (x86-64):

- Maksimum virtual memory: 256 TB per proses

Secara praktik pada sistem ini:

- RAM fisik: 3.7 GB
- Swap: 1 GB

Virtual memory yang bisa dialokasikan jauh lebih besar dari RAM fisik

BAB V

PENUTUP

5.1 KESIMPULAN

Praktikum manajemen memori ini telah berhasil mendemonstrasikan konsep-konsep fundamental sistem operasi melalui implementasi langsung dan analisis empiris. Hasil eksperimen menunjukkan bahwa setiap algoritma dan struktur data memiliki trade-off unik antara kecepatan, efisiensi ruang, dan kompleksitas implementasi. Algoritma Best-Fit terbukti paling efisien mengurangi fragmentasi eksternal, sementara LRU menghasilkan performa paging terbaik untuk pola akses dengan locality of reference. Implementasi multi-level page table menyeimbangkan kebutuhan memori dan kecepatan translasi, sedangkan inverted page table optimal untuk sistem dengan ruang alamat virtual besar. Praktikum ini juga mengonfirmasi keberadaan Belady's Anomaly pada algoritma FIFO, sekaligus menunjukkan pentingnya pemilihan strategi swapping yang tepat untuk menghindari thrashing.

5.2 SARAN

Berdasarkan hasil praktikum, disarankan untuk melakukan optimasi sistem dengan mempertimbangkan karakteristik beban kerja spesifik. Untuk sistem dengan permintaan memori dinamis, kombinasi algoritma Best-Fit dengan LRU dapat memberikan keseimbangan optimal antara utilisasi memori dan performa paging. Penggunaan multi-level page table dengan TLB yang memadai direkomendasikan untuk sistem 64-bit, sementara inverted page table cocok untuk embedded system dengan resource terbatas. Penambahan memori fisik dan optimasi ukuran swap space dapat mengurangi frekuensi swapping dan meningkatkan responsivitas sistem. Penelitian lanjutan dapat difokuskan pada algoritma hybrid dan machine learning untuk adaptive memory management yang responsif terhadap perubahan pola akses aplikasi modern.

DAFTAR PUSTAKA

Abraham Silberschatz, et al. "Operating System Concepts", 10th Edition

Andrew S. Tanenbaum. "Modern Operating Systems", 4th Edition